

Contents

1 Basic Statistics *5*

1.1 Mean *5*

1.2 Variance *6*

1.3 Median *8*

1.4 Histograms and Bell Curves *9*

1.5 Root-Mean-Squared *11*

2 Basic Statistics in Spreadsheets *15*

2.1 The Barrel Problem *15*

2.2 Solving It Symbolically *15*

2.3 Your First Spreadsheet *16*

2.4 Formatting *18*

2.5 Comma-Separated Values *19*

2.6 Statistics in Spreadsheets *20*

2.7 Histogram *21*

2.8 The Return of the Barrel Problem *22*

2.9 Graphing *25*

2.10 Other Things You Should Know About Spreadsheets *26*

2.11 Challenge: Make a spreadsheet *26*

3 Introduction to Python *29*

3.1 Getting Started with Python *29*

3.1.1 The Console and Running Python Programs *30*

3.1.2 Running a Python File *30*

3.2 Output `print` 31
3.3 Datatypes and Variables 32
3.4 Operations 35
3.4.1 Arithmetic Operations 35
3.4.2 Augmented Assignment Operators 36
3.4.3 Summary 36
3.5 Input and Output 37
3.5.1 Getting User Input 37
3.5.2 Input Always Returns a String 38
3.5.3 Casting Input Values 38
3.5.4 Common Input Errors 38
3.5.5 Summary 39
3.6 Conditionals, Loops, and Match-Case 40
3.6.1 Conditional Statements 40
3.6.2 Boolean Expressions 40
3.6.3 Loops 41
3.6.4 For Loops 41
3.6.5 While Loops 42
3.6.6 Breaking and Continuing Loops 42
3.6.7 Match-Case Statements 43
3.6.8 Summary 43
3.7 Lists and Strings 44
3.7.1 Indexing Strings 45
3.7.2 Length of a String 45
3.7.3 Iterating Over Strings 46
3.7.4 String Methods 46
3.7.5 Lists 46
3.7.6 Indexing Lists 47
3.7.7 Modifying Lists 49
3.7.8 List Length and Iteration 50
3.7.9 Common List Methods 50
3.7.10 Strings vs Lists 50
3.7.11 Summary 51
3.8 Is there more? 52

4 Sets and Logic 53

4.1 Sets 54
4.1.1 And and Or 55

4.1.2 How simple are sets? 55
4.1.3 Subsets 56
4.1.4 Union and Intersection of Sets 56
4.1.5 Venn Diagrams 57
4.2 Logic 59
4.3 Implies 59
4.4 If and Only If 59
4.5 Not 60
4.6 Cardinality 61
4.7 Complement of a Set 61
4.8 Subtracting Sets 62
4.9 Power Sets 63
4.10 Booleans in Python 63
4.11 The Contrapositive 65
4.12 The Distributive Property of Logic 65
4.13 Exclusive Or 65

5 Proofs ***67***
5.1 Introduction 67
5.2 Trivial Proof 68
5.3 Direct Proof 68
5.4 Proof By cases 68
5.5 Disproof By Counter-example 70
5.6 Proof By Contraposition 70
5.7 Proof By Contradiction 71
5.8 Proof By Induction 72
5.9 Summary 73

6 Introduction to Electricity ***75***
6.1 Units 76
6.2 Capacitors 77
6.3 Circuit Diagrams 78
6.4 Ohm's Law and Resistance 79
6.5 Power and Watts 80
6.6 Another great use of RMS 81
6.7 Electricity Dangers 82

7 DC Circuit Analysis ***85***
7.1 Resistors in Series 85

7.1.1 Kirchhoff's Voltage Law *86*
7.2 Resistors in Parallel *88*
7.3 Capacitors in series and parallel *89*
7.4 Kirchhoff's Current Law *90*
7.5 Summary *91*

A Answers to Exercises ***93***

Index *99*

CHAPTER 1

Basic Statistics

You live near a freeway, and someone asks you, "How fast do cars on that freeway drive?"

You say, "Pretty fast."

They reply, "Can you be more specific?"

So, you pull out your radar gun you happen to always keep on you, and tell them, "That one is going 32.131 meters per second."

To which they say, "I don't want to know about that specific car. I want to know about all the cars."

So, you spend the day beside the freeway measuring the speed of every car that goes by. And you get a list of a thousand numbers, including:

30.462 m/s	29.550 m/s	29.227 m/s
37.661 m/s	27.899 m/s	28.113 m/s
24.382 m/s	35.668 m/s	43.797 m/s
31.312 m/s	37.637 m/s	30.891 m/s

There are 12 numbers here. We say that there are 12 *samples*.

1.1 Mean

We often talk about the *average* of a set of samples, which is the same as the *mean*. To get the mean, sum up the samples and divide that number by the number of samples.

The numbers in that table sum to 388.599. If you divide that by 12, you find that the mean of those samples is 32.217 m/s.

We typically use the greek letter μ ("mu") to represent the mean.

Definition of Mean

If you have a set of samples $x_1, x_2, \ldots, x_n$, the mean is:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

This may be the first time you are seeing a summation ($\sum$). The equation above is equivalent to:

$$\mu = \frac{1}{n}(x_1 + x_2 + \ldots + x_n)$$

Exercise 1 Mean Grade

Working Space

Teachers often use the mean for grading. For example, if you took six quizzes in a class, your final grade might be the mean of the six scores. Find the mean of these six grades: 87, 91, 98, 65, 87, 100.

Answer on Page 93

If you tell your friend, "I measured the speed of 1000 cars, and the mean is 31.71 m/s", your friend will wonder, "Are most of the speeds clustered around 31.71? Or are they all over the place and just happen to have a mean of 31.71?" To answer this question, we use variance.

1.2 Variance

Definition of Variance

If you have n samples $x_1, x_2, \ldots, x_n$ that have a mean of μ, the *variance* is defined to be:

$$v = \frac{1}{n}\sum_{i=1}^{n} (x_i - \mu)^2$$

That is, you figure out how far each sample is from the median, you square that, and then you take the mean of all those squared distances.

x	$x - \mu$	$(x - \mu)^2$
30.462	-1.755	3.079
29.550	-2.667	7.111
29.227	-2.990	8.938
37.661	5.444	29.642
27.899	-4.318	18.642
28.113	-4.104	16.839
24.382	-7.835	61.381
35.668	3.451	11.912
43.797	11.580	134.106
31.312	-0.905	0.818
37.637	5.420	29.381
30.891	-1.326	1.757
$\sum x = 386.599$		$\sum (x - \mu)^2 = 323.605$
mean = 32.217		variance = 26.967

Thus, the variance of the 12 samples is 26.967. The bigger the variances, the farther the samples are spread apart; the smaller the variances, the closer samples are clustered around the mean.

Notice that most of the data points deviate from the mu by 1 to 5 m/s. Isn't it odd that the variance is a big number like 26.967? Remember that it represents the average of the squares. Sometimes, to get a better feel for how far the samples are from the mean, we use the square root of the variance, which is called *the standard deviation*.

The standard deviation of your 12 samples would be $\sqrt{26.9677} = 5.193$ m/s.

The standard deviation is used to figure out if a data point is an outlier. For example, if you are asked, "That car that just sped past. Was it going freakishly fast?" You might respond, "No, it was within a standard deviation of the mean." or "Yes, its speed was 2 standard deviations more than the mean. They will probably get a ticket."

A singular μ usually represents the mean. σ usually represents the standard deviation. So σ^2 represents the variance.

Exercise 2 Variance of Grades

Working Space

Now, find the variance for your six grades. As a reminder, they were: 87, 91, 98, 65, 87, 100.

What is your standard deviation?

Answer on Page 93

1.3 Median

Sometimes you want to know where the middle is. For example, you want to know the speed at which half the cars are going faster and half are going slower. To get the median, you sort your samples from smallest to largest. If you have an odd number of samples, the one in the middle is the median. If you have an even number of samples, we take the mean of the two numbers in the middle.

In our example, you would sort your numbers and find the two in the middle:

24.382
27.899
28.113
29.227
29.550
30.462
30.891
31.312
35.668
37.637
37.661
43.797

You take the mean of the two middle numbers: $(30.462 + 30.891)/2 = 30.692$. The median speed would be 30.692 m/s.

Medians are often used when a small number of outliers majorly skew the mean. For example, income statistics usually use the median income because a few hundred billion-

aires raise the mean a great deal.

Exercise 3 Median Grade

Working Space

Find the median of your six grades: 87, 91, 98, 65, 87, 100.

Answer on Page 93

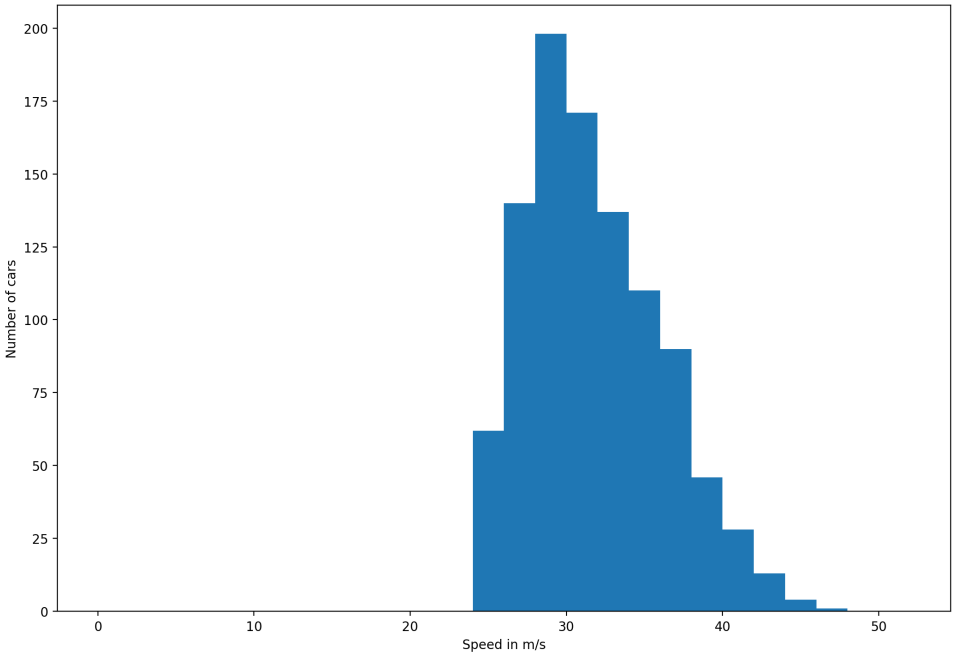
1.4 Histograms and Bell Curves

A histogram is a bar chart that shows how many samples are in each group. In our example, we group cars by speed. Maybe we count the number of cars going between 30 and 32 m/s. Next, we count the cars going between 32 and 34 m/s. Finally, we make a bar chart from that data.

Your 1000 cars would break up into these groups:

0 - 2 m/s	0 cars
2 - 4 m/s	0 cars
4 - 6 m/s	0 cars
...	...
20 - 22 m/s	0 cars
22 - 24 m/s	0 cars
24 - 26 m/s	65 cars
26 - 28 m/s	160 cars
28 - 30 m/s	175 cars
30 - 32 m/s	168 cars
32 - 34 m/s	150 cars
34 - 36 m/s	114 cars
36 - 38 m/s	79 cars
38 - 40 m/s	52 cars
40 - 42 m/s	20 cars
42 - 44 m/s	12 cars
44 - 46 m/s	4 cars
46 - 48 m/s	1 cars
48 - 50 m/s	0 cars

Next, we make a bar chart from that:



Often, a histogram will tell the story of the data. Here, you can see that no one is going less than 24 m/s, but a lot of people travel at 30 m/s. There are a few people who travel over 40 m/s, but there are also a couple of people who drive much faster than anyone else.

If we look closely at the shape of our histogram, we notice something interesting: it has a smooth, rounded hill in the middle and it tapers off on both sides. This is a common pattern in statistics, and it often suggests that the data follows what's called a **normal distribution**, also known as a **bell curve**.

A **bell curve** is a continuous curve that models how data is distributed when:

- Most values are near the average (mean), located at the peak of the curve,
- Fewer values are far from the mean, the tails of the curve,
- The distribution is roughly symmetric.

In our case, the majority of cars are traveling around 30–32 m/s. There are fewer cars going slower or faster than that. If we collected even more data (say, 10,000 cars), the histogram would start to resemble a smooth bell curve even more closely. The units of the x-axis (speed) would still be the same, but the y-axis would represent the *probability density* of finding a car at a certain speed, rather than just the count of cars in each speed

range.

The peak of the bell curve represents the **mean speed**, and the spread of the curve is related to the **standard deviation** — a measure of how spread out the speeds are. Most data falls within:

- 1 standard deviation of the mean ($\mu \pm \sigma$): about 68% of the cars,
- 2 standard deviations: about 95%,
- 3 standard deviations: about 99.7%.

This means that if we know the mean speed and the standard deviation, we can predict how many cars will be within certain speed ranges. If you have ever heard of Six Sigma methodology, that means data falls within six standard deviations of the mean, which is a very high level of quality control (3.4 defects per million measurements!).

So when we say a dataset "looks like a bell curve," we mean that it has a predictable and symmetric structure, often meaning it is a reliable set of data with few outliers and consistent results.

1.5 Root-Mean-Squared

Scientists have a mean-like statistic that they love. It is named quadratic mean, but most just calls it Root-Mean-Squared, or RMS.

Definition of RMS

If you have a list of numbers $x_1, x_2, \ldots, x_n$, their RMS is

$$\sqrt{\frac{1}{n}\left(x_1^2 + x_2^2 + \ldots + x_n^2\right)}$$

You are taking the square root of the mean of squares of the samples, thus the name Root-Mean-Squared.

Using your 12 samples:

x	x^2
30.462	927.933
29.550	873.203
29.227	854.218
37.661	1418.351
27.899	778.354
28.113	790.341
24.382	594.482
35.668	1272.206
43.797	1918.177
31.312	980.441
37.637	1416.544
30.891	954.254
Mean of x^2	1064.875
RMS	32.632

Why is RMS useful? Let's say that all cars had the same mass m, and you need to know what the average kinetic energy per car is. If you know the RMS of the speeds of the cars is v_{rms}, the average kinetic energy for each car is

$$k = \frac{1}{2}mv_{rms}^2$$

(You don't believe me? Let's prove it. Substitute in the RMS:

$$k = \frac{1}{2}m\sqrt{\frac{1}{n}\left(x_1^2 + x_2^2 + \ldots + x_n^2\right)}^2$$

The square root and the square cancel each other out:

$$k = \frac{1}{2}m\frac{1}{n}\left(x_1^2 + x_2^2 + \ldots + x_n^2\right)$$

Use the distributive property:

$$k = \frac{1}{n}\left(\frac{1}{2}mx_1^2 + \frac{1}{2}mx_2^2 + \ldots + \frac{1}{2}mx_n^2\right)$$

That is all the kinetic energy divided by the number of cars, which is the mean kinetic energy per car. Quod erat demonstrandum! (That is a Latin phrase that means "which is what I was trying to demonstrate". You will sometimes see "QED" at the end of a long mathematic proof.)

Now you are ready for the punchline: Kinetic energy and heat are the same thing. Instead of cars, heat is the kinetic energy of molecules moving around. More on this soon.

Video: Mean, Median, Mode: https://www.youtube.com/watch?v=5C9LBF3b65s

CHAPTER 2

Basic Statistics in Spreadsheets

When you completed the problems in the last section, you probably noticed how long it took to compute statistics like the mean, median, and variance by hand. Luckily, computers were designed to free us from these sorts of tedious tasks. The most basic tool for automating calculations is the spreadsheet program.

The first spreadsheet program (VisiCalc) was introduced in 1979 as a tool for finance people to play "what if" games. For example, a company might make a spreadsheet that told them how much more profit they would make if they changed from using an expensive metal to using a cheaper alloy.

There are lots of spreadsheet programs, including Microsoft's Excel, Google Sheets, and Apple's Numbers. Any spreadsheet program will work; they are all very similar. The instructions and screenshots here will be from Google Sheets — a free spreadsheet program you use through your web browser.

2.1 The Barrel Problem

In honor of this history, let's start by studying a business question: You have a friend who dreams of quitting her job to become a cooper. (A cooper makes barrels that are used for aging wine and whiskey.) According to her:

- It costs $45 dollars in materials to build one barrel.
- A barrel sells for $100 dollars.
- The workshop/warehouse she wants to rent costs $2000 per month.
- Taxes take 20% of her profits.
- She needs to make $4000 monthly after taxes.

She has asked you, "How many barrels do I need to make each month?"

2.2 Solving It Symbolically

Many problems can be solved two ways: symbolically or numerically. To solve this problem symbolically, you would write out the facts as equations or inequalities, then do

symbol manipulations until you ended up with an answer. In this case, you would let b be the number of barrels and create the following inequality:

$$(1.0 - 0.2)\,(b(100 - 45) - 2000) \geq 4000$$

You would simplify it:

$$(0.8)\,(55b - 2000) \geq 4000$$

And simplify it more:

$$44b - 1600 \geq 4000$$

If that is true, then:

$$44b \geq 5600$$

And if that is true, then:

$$b \geq \frac{1400}{11}$$

1400/11 is about 127.27, so she needs to make and sell 128 barrels each month.

That is a perfect answer, and we didn't need a spreadsheet at all. However:

- As problems get larger and more realistic, it gets much more difficult to solve them symbolically.
- As soon as you say "Yes, you need to make and sell 128 barrels each month." Your friend will ask "What if I make and sell 200 barrels? How much money will I make then?"

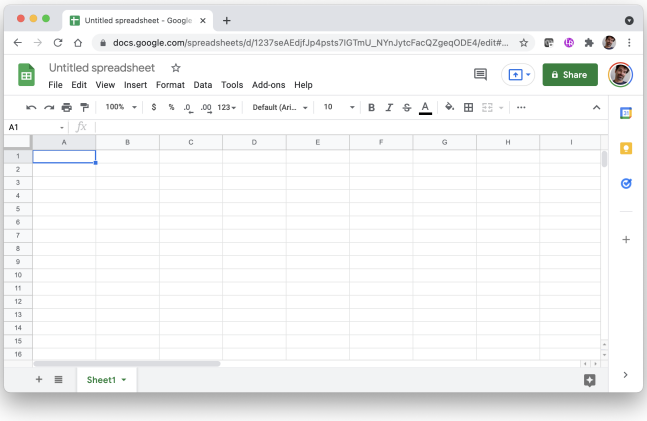
So, we use a spreadsheets to solve the problem numerically.

2.3 Your First Spreadsheet

Let's first make an initial spreadsheet.

In whatever spreadsheet program you are using, create a new spreadsheet document.

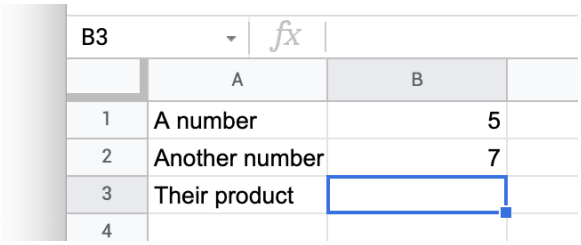
A spreadsheet is essentially a grid of cells. In each cell, you can put data (like numbers or text) and formulas.



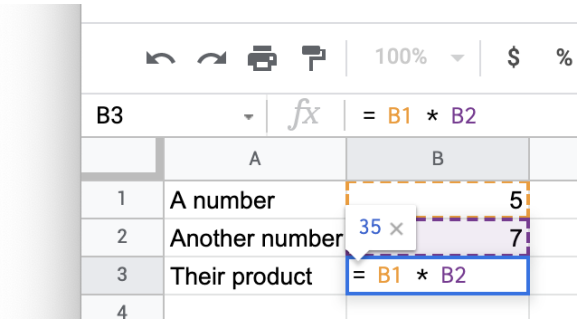
Let's put some labels in the column:

- Select the first cell (A1) and type "A number".
- Select the cell below it (A2) and type "Another number".
- Select the cell below that one (A3) and type "Their product".
- In the next column, type the number 5 in B1 and 7 in B2.

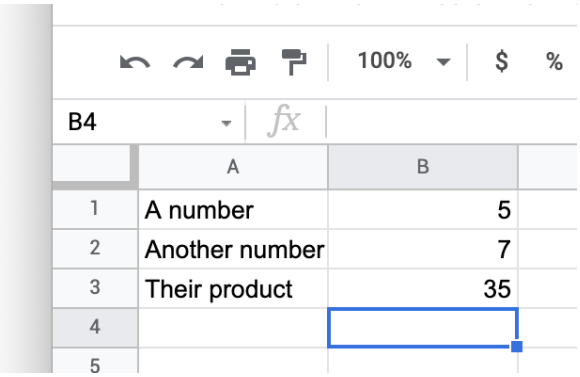
It should look like this:



Now, put a formula in cell B3. Select B3, and type "= B1 * B2". The spreadsheet knows this is a formula because it starts with '='. It will look like this as you type:



When you press Return or Tab, the spreadsheet will remember the formula, but display its value:



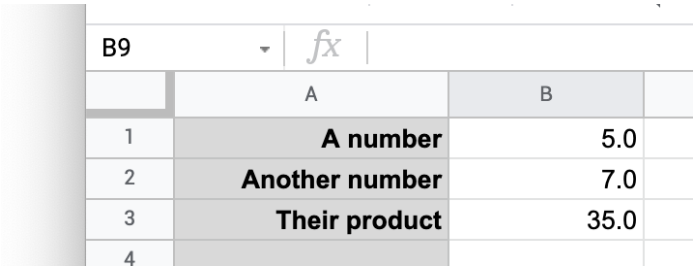
If you change the values of cell B1 or B2, the cell B3 will automatically be recalculated. Try it.

2.4 Formatting

Every spreadsheet lets you change the formatting of your columns and cells. They are all a little different, so play with your spreadsheet a little now. Try to do the following:

- Set the background of the first column to light gray.
- Right-justify the text in the first column.
- Make the text in the first column bold.
- Make the numbers in the second column have one digit after the decimal point.

It should look something like this:



That's a spreadsheet. You have a grid of cells, each of which can hold a value or a formula that uses values from other cells. The cells containing formulas automatically update as you edit the values in the other cells.

2.5 Comma-Separated Values

A large amount of data is exchanged in a file format called *Comma-Separated Values* or just CSV. Each CSV file holds one table of data. It is a text file, and each line of text corresponds to one row of data in the table. The data in each column is separated by a comma. The first line of a CSV is usually the names of the columns. A CSV might look like this:

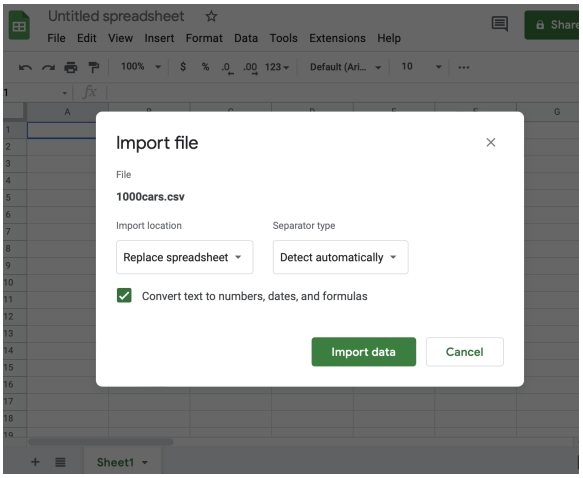
```
studentID,firstName,lastName,height,weight
1,Marvin,Sumner,260,45.3
2,Lucy,Harris,242,42.2
3,James,Boyd,261,44.2
```

In your digital resources for this module, you should have a file called `1000cars.csv`. It is a CSV with only one column called "speed". The first few lines look like this:

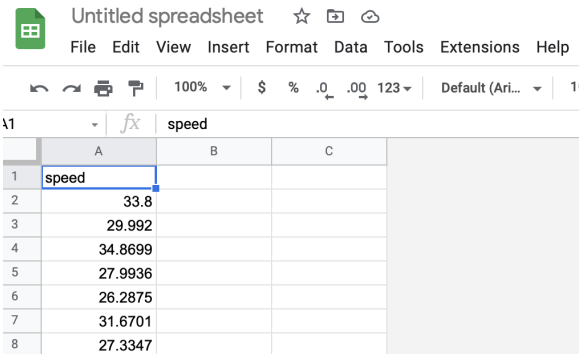
```
speed
33.8000
29.9920
34.8699
27.9936
```

There is a title line and 1000 data lines.

Import this CSV into your spreadsheet program. In Google Sheets, it looks like this:



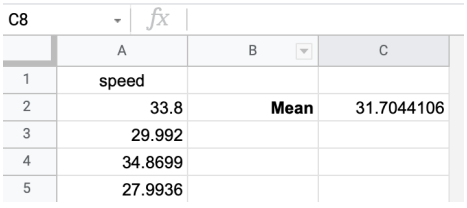
You should see a long, long column of data appear. (Mine goes from cell A2 through A1001.)



2.6 Statistics in Spreadsheets

Let's take the mean of all 1000 numbers. In cell B2, type in a label: "Mean". (Feel free to format your labels as you wish. Bolding is recommended.)

In cell C2, enter the formula "=AVERAGE(A2:A1001)". When you press return, the cell will show the mean: 31.70441, if done correctly.



Notice that by specifying that the function `AVERAGE` was to be performed on a range of cells: cells A2 through (":") A1001.

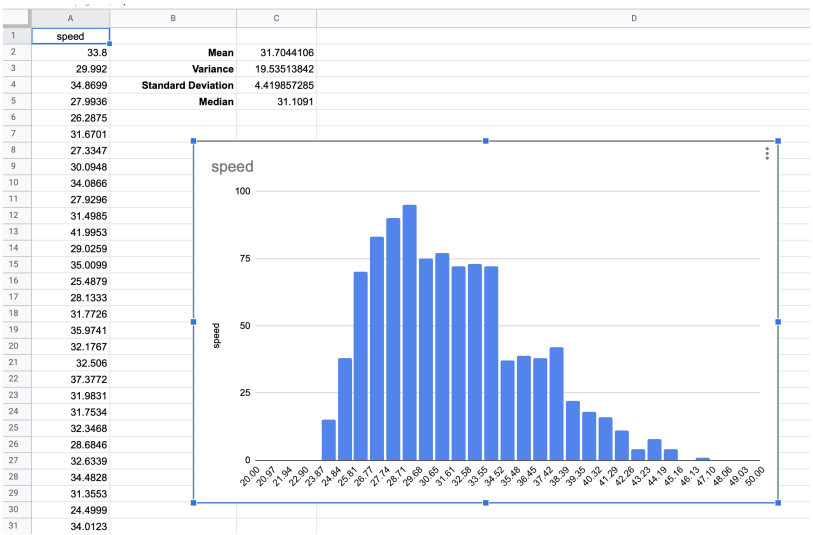
Do the calculations for variance, standard deviation, and median.

- The function for variance is `VAR`.
- The function for standard deviation is `STDEV`.
- The function for median is `MEDIAN`.

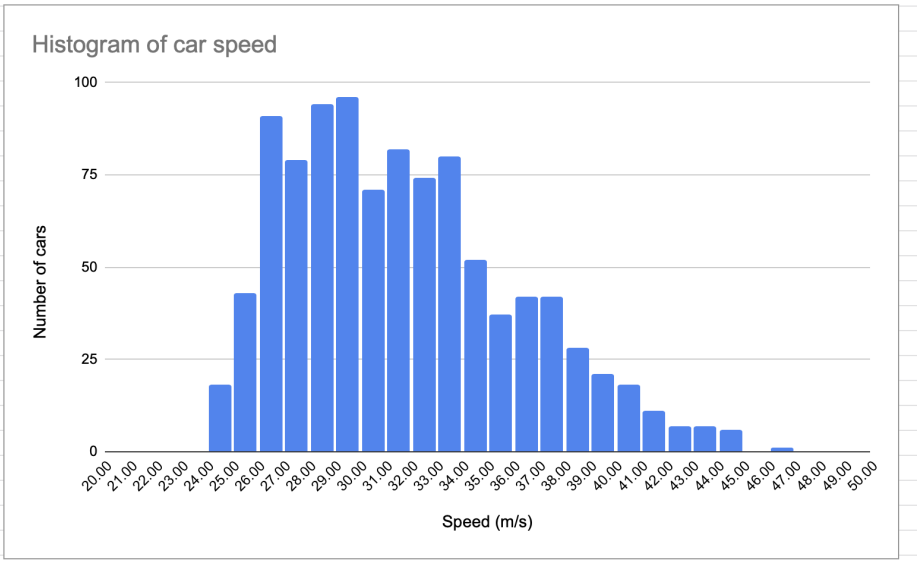
	A	B	C
1	speed		
2	33.8	**Mean**	31.7044106
3	29.992	**Variance**	19.53513842
4	34.8699	**Standard Deviation**	4.419857285
5	27.9936	**Median**	31.1091
6	26.2875		
7	31.6701		

2.7 Histogram

Most spreadsheets have the ability to create a histogram. In Google Sheets, you select the entire range A2:A1001 by selecting the first cell and then shift-clicking the last. Next, you choose Insert→Chart. In the inspector, change the type of the chart to a histogram (at the bottom under "other"). This will get you a basic histogram.



Play with the formatting to see how unique you can make data. Here is an example:



Exercise 4 RMS

Working Space

In your spreadsheet, calculate the quadratic mean (the root-mean-squared) of the speeds.

You will need the following three functions:

- `SUMSQ` returns the sum of the squares of a range of cells.
- `COUNT` returns the number of cells in a range that contains numbers.
- `SQRT` returns the square root of a number.

Answer on Page 93

2.8 The Return of the Barrel Problem

Let's get back to our example. Put labels in the A column:

- Barrels produced (per month)
- Materials cost (per barrel)
- Sale price (per barrel)
- Pre-tax earnings (per month)
- Taxes (per month)
- Take home pay (per month)

Format them any way you like. It should look something like this:

	A
1	**Barrels Produced** (per month)
2	**Materials cost** (per barrel)
3	**Sale price** (per barrel)
4	**Rent** (per month)
5	**Pretax Earnings** (per month)
6	**Taxes** (per month)
7	**Take home pay** (per month)
8	

In the B column, the first four cells are values (not formulas):

- 115 formatted as a number with no decimal point
- 45 formatted as currency
- 100 formatted as currency
- 2000 formatted as currency

It should look something like this:

	A	B
1	**Barrels Produced** (per month)	115
2	**Materials cost** (per barrel)	$45.00
3	**Sale price** (per barrel)	$100.00
4	**Rent** (per month)	$2,000.00

The next three cells in the B column will have formulas:

- B1 * (B3 - B2) - B4
- 0.2 * B5

- B5 - B6

It should look something like this:

	A	B
1	**Barrels Produced** (per month)	115
2	**Materials cost** (per barrel)	$45.00
3	**Sale price** (per barrel)	$100.00
4	**Rent** (per month)	$2,000.00
5	**Pretax Earnings** (per month)	$4,325.00
6	**Taxes** (per month)	$865.00
7	**Take home pay** (per month)	$3,460.00
8		

Now you can share this spreadsheet with your friend, and she can put different values into the cells for what-if games, such as "If I can get my materials cost down to $42 per barrel, what happens to my take home pay?"

Sometimes it is nice to show a range of values for a variable or two. In this case, it might be nice to show your friend what the numbers look like if she produces 115, 120, 125, 130, 135, or 140 barrels per month.

We have one column, and now we need six. How do we duplicate cells?

1. Click B1 to select it, then shift-click on B7 to select all seven cells.
2. Copy them. (Depending on what program you are using, there is likely a menu item for this.)
3. Click C1 to select it
4. Paste them.

	A	B	C
1	**Barrels Produced** (per month)	115	115
2	**Materials cost** (per barrel)	$45.00	$45.00
3	**Sale price** (per barrel)	$100.00	$100.00
4	**Rent** (per month)	$2,000.00	$2,000.00
5	**Pretax Earnings** (per month)	$4,325.00	$4,325.00
6	**Taxes** (per month)	$865.00	$865.00
7	**Take home pay** (per month)	$3,460.00	$3,460.00
8			

We want the first cell in the new column to be 120. You could just type in 120, but let's do something more clever. Put a formula into that cell: = B1 + 5. Now, the cell should show 120.

Why did we put in a formula? When we duplicate this column, this cell will always have 5 more barrels than the cell to its left.

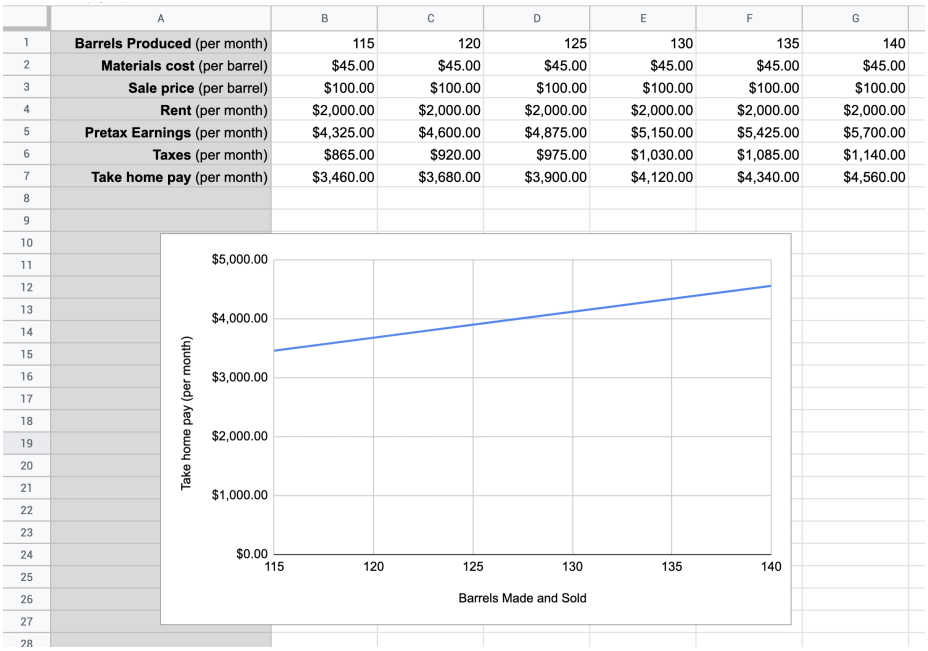
Next, let's duplicate the second column a few times. The easy way to do this is to select the cells as you did before, then drag the lower-right corner to the right until column G is in the selection. When you end the drag, the copies will appear:

	A	B	C	D	E	F	G
1	**Barrels Produced** (per month)	115	120	125	130	135	140
2	**Materials cost** (per barrel)	$45.00	$45.00	$45.00	$45.00	$45.00	$45.00
3	**Sale price** (per barrel)	$100.00	$100.00	$100.00	$100.00	$100.00	$100.00
4	**Rent** (per month)	$2,000.00	$2,000.00	$2,000.00	$2,000.00	$2,000.00	$2,000.00
5	**Pretax Earnings** (per month)	$4,325.00	$4,600.00	$4,875.00	$5,150.00	$5,425.00	$5,700.00
6	**Taxes** (per month)	$865.00	$920.00	$975.00	$1,030.00	$1,085.00	$1,140.00
7	**Take home pay** (per month)	$3,460.00	$3,680.00	$3,900.00	$4,120.00	$4,340.00	$4,560.00

Nice, right? Now your friend can easily see how many barrels correspond to how much take-home pay. But do you know what would be even more helpful? A graph.

2.9 Graphing

Graphing is a little different on every different platform. Here is what you want the graph to look like:



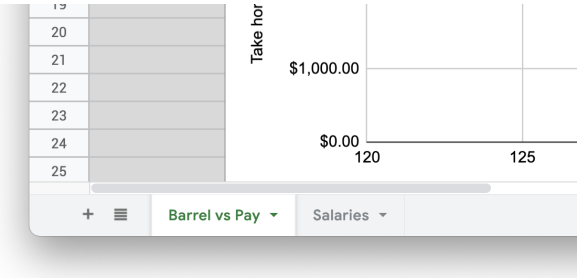
On Google Sheets:

1. Select cells B7 through G7.
2. Choose the menu item Insert -> Chart.
3. Choose the chart type (Line)

4. Add the X-axis to be B1 through G1.
5. Under the Customize tab, Set the label for the X-axis to be "Barrels Made and Sold".
6. Delete the chart title (which is the same as the Y-axis label).

2.10 Other Things You Should Know About Spreadsheets

Your spreadsheet document can have several "Sheets". Each has its own grid of cells. The sheet has a name; usually, you call it something like "Salaries". When you need to use a value from the "Salaries" sheet in another sheet, you can specify "Salaries!A2" — that is, cell A2 on sheet "Salaries". To flip between the sheets, there is usually a tab for each at the bottom of the document.



By default, the cell references are relative. In other words, when you write a formula in cell H5 that references the value in cell G4, the cell remembers "The cell that is one up and one to the left of me." Thus, if you copy that formula into B9, now that formula reads the value from A8.

If you want an absolute reference, you use $. If H5 references G4, G4 will be used no matter where on the sheet the formula is copied to.

You can use the $ on the row or column. In $A4, the column is absolute and the row is relative. In A$4, the row is absolute and the column is relative.

2.11 Challenge: Make a spreadsheet

You have a company that bids on painting jobs. Make a spreadsheet to help you do bids. Here are the parameters:

- The client will tell you how many square meters of wall needs to be painted.
- Paint costs $0.02 per square meter of wall
- On average, a square meter of wall takes 0.02 hours to paint.

- You can hire painters at $15 per hour.
- You add 20% to your estimated costs for a margin of error and profit.

Make a spreadsheet such that when you type in the square meters to be painted, the spreadsheet tells you how much you will spend on paint and labor. It should also tell you what your bid should be.

Chapter 3

Introduction to Python

In this chapter we will discuss the very basics of Python. No we are not talking about snake anatomy, but the programming language Python. Python is a high-level, interpreted programming language known for its readability and versatility. It is widely used in various fields such as web development, data analysis, artificial intelligence, scientific computing, and more. It can be used for data parsing, visualization, and even machine learning. Python's syntax is designed to be easy to read and write, making it an excellent choice for beginners and experienced programmers alike. Throughout this chapter, we encourage you to write out the provided lines of code yourself! This will reinforce skills like computer literacy, Python syntax and troubleshooting, and of course, typing speed.

3.1 Getting Started with Python

To get started with Python, you need to have it installed on your computer. You can download the latest version of Python from the official website: `https://www.python.org/downloads/`. Follow the installation instructions for your operating system.

Once you have python installed, you can write and run Python code using various methods:

- **Interactive Shell:** You can open a terminal or command prompt and type `python` or `python3` to start an interactive Python shell where you can type and execute Python commands line by line.
- **Script Files:** You can write your Python code in a text file with a `.py` extension and run it using the command `python filename.py` or `python3 filename.py` in the terminal.
- **Integrated Development Environments (IDEs):** There are several IDEs available for Python, such as PyCharm, VSCode, and Jupyter Notebooks, which provide a more user-friendly environment for writing and running Python code. We will assume you will use VSCode, as it is widely used among computer scientists and developers around the world, and has direct integration to GitHub.

We are going to install VSCode as our IDE for this course. You can download it from `https://code.visualstudio.com/`. After installing VSCode, you will also need to install the Python extension for VSCode, which can be found in the Extensions Marketplace within VSCode.

3.1.1 The Console and Running Python Programs

When working with Python, it is important to understand the difference between *writing code* and *executing code*. Writing code simply means typing instructions into a file using a text editor or IDE. Executing code means telling Python to read those instructions and perform them.

The *console*, also called the *terminal* or *command prompt*, is a text-based interface where you can run programs and view their output. Any text printed using the `print()` function will appear in the console.

When you run a Python program, Python reads your file from top to bottom and executes each line in order. This is an important idea that will come up often: **Python code is executed sequentially, one line at a time**.

3.1.2 Running a Python File

To run a Python file, you must use the console. In VSCode, you can open the integrated terminal by selecting `Terminal` → `New Terminal` from the menu. Make sure your terminal is open in the same folder as your Python file.

If your file is named `hello.py`, you can run it by typing:

```
python hello.py
```

or, on some systems:

```
python3 hello.py
```

After pressing Enter, Python will execute the file and display any output in the console. If there are errors in your code, Python will stop execution and display an error message instead. You may also see a green play button at the top of your window, you could also run your program with that!

At this point, it is helpful to remember:

- Writing code does nothing by itself
- Code only runs when you explicitly execute the file
- Output from `print()` appears in the console

3.2 Output `print`

Let's talk more about the `print()` statement. It can take a string, multiple variables, or a combination of strings and variables. We are going to introduce the syntax first, and then expand on the definitions later.

```
x = "This is a variable."
print("This is a string.", x)
```

This should output `This is a string. This is a variable`. You can also do multiple variables:

```
x = "Good"
y = "Morning,"
z = "Chicago"
print(x, y, z)
```

This will output `Good Morning, Chicago`. Alternatively, printing can have addition within it:

```
x = "Good"
y = "Morning,"
z = "Chicago"
print(x+y+z)
```

This will output `GoodMorning,Chicago`. Notice the difference!

If you want to combine numbers and strings,

```
x = "it is"
y=78
z="degrees outside"
print(x+y+z)
```

This will not work! Instead, we should use commas:

```
x = "it is"
y=78
z="degrees outside"
print(x, y, z)
```

This will correctly output "it is 78 degrees outside"

By default, a space is added in between each argument (where the commas would be).

We can change what separates each variable by the `sep=` argument.

```
x = "it is"
y=78
z="degrees outside"
print(x, y, z, sep="..!..")
```

This will output `it is..!..78..!..degrees outside`

By default, Python includes a new line at the end of each print statement. We can alter this to anything else by using the `end=` argument:

```
print("Good Morning, Chicago.", end="!!")
```

Outputting `Good Morning, Chicago.!!` to the console, with no new line character inserted. Note that the next print statement will continue on the same line unless it contains a new line character.

```
print("Hello World!", end=" ")
print("I will print on the same line.")
```

We have used all this terminology like strings and variables a lot, let's dive a bit deeper into it!

3.3 Datatypes and Variables

There are many different datatypes in Python, and all of them can be passed to `print` function. Here's the main few:

Strings Strings are just a sequence of characters. Anything closed between quotes gets included in the string, such as "This is the Python Chapter for the Kontinua Sequence!!%& × ()-+=1234567890 &"

Integers Integers are any whole number, positive negative or 0. Theoretically, integer values can go on for infinity, but it is important to understand they take up space in computer memory.

Floats Floats, or floating point values, are numbers that include decimal places. Division can be done between integers and floats but may result in a different value.

Booleans Booleans only two values: `True` or `False`. The output of conditional statements

(such as if's or while's) are booleans. They only answer Yes and No questions, and are important on deciding between two possible paths in code.

Sequence Types Sequences are consecutive lists or pairs of other data types can be represented in various forms: `list`, `set`, and `tuple`. Values are accessed by positions.

Mapping Types The main mapping type is a `dict`, short for dictionary. Mapping types store values as key–value pairs instead of positions. You access items using a key, not an index.

Let's create some of these in our code:

```
x = 5
y = 5.6
z = True
a = [x, y, z]
```

A *variable* is a container for information, usually containing one of the datatypes established above. In Python, variables are assigned using the assignment operator =, with the name of the variable on the left side of the operator and the value it is assigned to on the right side.

Python has specific naming rules for variables. The name

- Must start with a letter or underscore
- Cannot contain spaces
- Are case-sensitive

In our code above, `x` is a *variable* containing the number 5, an Integer. `y` contains the float 5.6, while `z` is a boolean with the value `True`. We then created a list containing the variables `x`, `y`, and `z`.

A unique feature of Python is that variables can change their datatypes even after assignment. We can see the datatype of an object using the `type()`

```
x = 10
print(x)
print(type(x))

x = "Hello!"
print(x)
print(type(x))
```

Output:

```
10
<class 'int'>
Hello!
<class 'str'>
```

Notice that, because of that feature, the datatype of a variable is not defined beforehand. This is a core difference between Python and other programming languages like Java or C++, which we will explore in the future.

We can also define our floats using scientific notation.

```
x = 1e3     # 1000.0
y = 2.5e-4  # 0.00025
```

Exercise 5 Identify the variable type

Working Space

Below are 5 variables defined in Python. Identify their type as `str`, `float`, `bool`, `list`, or `int` by filling in the table below.

```
a = 10
b = 3.5
c = "10"
d = True
e = [1, 2, 3]
```

Variable	Value	Data Type
a	10	
b	3.5	
c	”10”	
d	True	
e	[1, 2, 3]	

Answer on Page 94

Exercise 6 What does this do?

What is the output of the following code?

Working Space

```
1 print(type(3.0) == type(3))
```

Answer on Page 94

3.4 Operations

Operations can be done on both variables and numbers alike. Most commonly they will be used for mathematical operations or simplifying operations of equivalence.

3.4.1 Arithmetic Operations

Math operations in Python are very similar to standard math operations. We need them for any type of data operation or math parsing.

```
1  a = 100
2  b = 3
3  c = 100.0
4  d = -100
5  print(a + b) # addition
6  print(a - b) # subtraction
7  print(a * b) # multiplication
8  print(a / b) # division (results in a float)
9  print(a % b) # modulus (remainder) operator - useful for division checks
10 print(a ** b) # exponentiation
11 print(a // b) # floor division
12 print(c // b) # floor division as a float
13 print(d // b) # negative flooring goes further DOWN
```

Output:

```
1 103
2 97
3 300
```

```
    33.333333333333336
    1
    1000000
    33
    33.0
    34
```

Note that division by `0` is not a valid operation, just as in algebra. This will cause an `ZeroDivisionError`.

3.4.2 Augmented Assignment Operators

You may want to do the following operation

```
x = 0
while True:
    print(x)
    x = x + 1
```

This would print the counting numbers, increasing by 1, forever. The `x = x + 1` rewrites the value of `x` equal to the current value of `x`, and adds one to it. There is a short hand for this kind of operation, called an *augmented assignment operator*, which combines assignment operators and operations. These are commonly used in loops and accumulation counters:

```
x = 10
x += 5   # same as x = x + 5
x -= 3   # same as x = x - 3
x *= 2   # same as x = 2 * x
x /= 4   # same as x = x / 4
```

Following order of operations, `x = 6` after all lines are executed.

3.4.3 Summary

- Math operations are done with `+`, `-`, `*`, and `/`
- Augmented Operations `+=`, `-=`, `*=`, `/=` are common for loops and counting operations.

Exercise 7 Tracing Chart

Let's create a tracing chart. What happens after each line of code is run?

Working Space

```
1 n = 5
2 m = "3"
3 n = n + 1
4 m = m + "1"
```

Line Executed	n (value, type)	m (value, type)
After line 2		
After line 3		
After line 4		

Answer on Page 94

3.5 Input and Output

So far, we have only displayed information using the `print()` function. While output is useful, most programs also need a way to receive information from the user. In Python, this is done using the built-in `input()` function.

3.5.1 Getting User Input

The `input()` function pauses the program and waits for the user to type something into the console. Whatever the user types is then returned and can be stored in a variable.

```
1 name = input("Enter your name: ")
2 print("Hello,", name)
```

In this example:

- The text inside `input()` is displayed as a prompt in the console
- The program waits for the user to type a response and press Enter

- The entered text is stored in the variable `name`
- The `print()` function then outputs a greeting using that value

3.5.2 Input Always Returns a String

A very important rule in Python is that `input()` **always returns a string**, even if the user types a number. This means that mathematical operations cannot be performed on input values unless they are converted to a numeric type.

Consider the following incorrect example:

```
age = input("Enter your age: ")
print(age + 1)
```

This code will result in an error, because Python cannot add a number to a string.

To perform calculations, the input must be *cast* to a different datatype.

3.5.3 Casting Input Values

Casting is the process of converting one datatype into another. Python provides built-in functions such as `int()`, `float()`, and `str()` for this purpose.

```
age = int(input("Enter your age: "))
print(age + 1)
```

Here:

- `input()` returns a string
- `int()` converts that string into an integer
- The program can now perform arithmetic operations

3.5.4 Common Input Errors

If the user enters something that cannot be converted to the requested datatype, Python raises a runtime error called a `ValueError`. For example:

```
age = int(input("Enter your age: "))
```

If the user types `hello` instead of a number, Python will produce an error similar to the following:

```
ValueError: invalid literal for int() with base 10
```

This error occurs because the string `"hello"` cannot be converted into an integer. Handling these types of errors will be discussed later when we introduce `try-except` statements.

3.5.5 Summary

- `print()` is used to display output to the console
- `input()` is used to receive text input from the user
- `input()` always returns a string
- Casting is required to convert input into numeric types

Exercise 8 User Input Types

Working Space

```
x = input("Enter a number: ")
print(x + 5)
```

What will happen when this code is run and the user enters `10`? If there is an issue, explain what it is, why it occurs, and how to fix it.

Answer on Page 94

3.6 Conditionals, Loops, and Match-Case

Most programs need to make decisions and repeat actions. Programmers do not want to write the same code multiple times, as this introduces *redundancy*. In Python, this is done using *conditionals* and *loops*. Conditionals allow the program to choose between different paths of execution, while loops allow code to be run multiple times.

3.6.1 Conditional Statements

Conditional statements execute code only if a given condition is true. Python uses the keywords `if`, `elif`, and `else` to define these conditions.

```
x = 10

if x > 0:
    print("x is positive")
elif x == 0:
    print("x is zero")
else:
    print("x is negative")
```

In this example:

- The expression after `if` is evaluated as a boolean
- If the condition is `True`, the indented block runs
- If it is `False`, Python checks the next condition. The `elif` block is the only way to check another branch after one is returned `False`. The `else` branch is a last resort, and only runs if all previous statements are false
- Only one branch of the conditional will execute

It is important to note that **indentation is required** in Python. All statements belonging to a conditional block must be indented at the same level.

3.6.2 Boolean Expressions

Conditions are built using comparison and logical operators. These expressions always evaluate to either `True` or `False`.

```
a = 5
b = 10

print(a < b)   # less than
print(a == b)  # equal to
print(a != b)  # not equal to
```

Logical operators can be used to combine conditions. Conditions can be *strung together* using `and`, `or`, and `not`

```
x = 7

if x > 0 and x < 10:
    print("x is between 1 and 9")
if x > 0 or x < -3:
    print("x is greater than 0 or less than -3")

y = False
print(not y)

```

3.6.3 Loops

Loops allow code to be repeated multiple times. Python provides two main types of loops: `for` loops and `while` loops.

To define a loop, Python uses the keywords `for` and `while`, followed by a condition or sequence to iterate over.

- the loop header, with correctly defined endpoints
- a colon :
- an indented block of code to iterate over

3.6.4 For Loops

A `for` loop is used to iterate over a sequence, such as a list or a range of numbers.

```
for i in range(5):
    print(i)
```

This code will print the numbers 0 through 4. The `range()` function generates a sequence of integers starting at 0 and stopping before the given value.

```
numbers = [10, 20, 30]

for n in numbers:
    print(n)
```

3.6.5 While Loops

A `while` loop repeats as long as a condition remains true.

```
x = 5

while x > 0:
    print(x)
    x -= 1
```

In this example:

- The condition is checked before each iteration
- The loop stops once the condition becomes `False`

Care must be taken to ensure that the condition eventually becomes false. Otherwise, the program will enter an infinite loop, one which never stops and the computer will run forever, risking hardware component issues.

3.6.6 Breaking and Continuing Loops

Python provides keywords to control loop execution.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

The `break` statement immediately exits the loop.

```python
for i in range(5):
    if i == 2:
        continue
    print(i)
```

The `continue` statement skips the current iteration and moves to the next one.

3.6.7 Match-Case Statements

Python also provides the `match-case` statement, which allows a value to be compared against several possible patterns. This is similar to a switch statement in other programming languages.

```python
command = input("Enter a command: ")

match command:
    case "start":
        print("Starting program")
    case "stop":
        print("Stopping program")
    case "pause":
        print("Pausing program")
    case _:
        print("Unknown command")
```

The underscore (`\_`) acts as a default case if no other patterns match.

3.6.8 Summary

- Conditionals allow programs to make decisions
- Boolean expressions evaluate to `True` or `False`
- Loops allow code to be repeated efficiently
- `for` loops iterate over sequences
- `while` loops repeat based on a condition
- `match-case` provides a structured way to handle multiple cases

Exercise 9 Predict the Output

Fill in the table with the outputs for the given inputs.

Working Space

```
x = int(input("Enter a number: "))

if x > 10:
    print("Large")
elif x == 10:
    print("Ten")
else:
    print("Small")
```

Input	Output
5	
10	
15	

Answer on Page 95

Exercise 10 Fill in the Blanks: Loop Edition

Fill in the following code to print all even numbers from 0 to 20 (inclusive).

Working Space

```
for i in __________:
    print(__________)
```

Answer on Page 95

3.7 Lists and Strings

We talked about creating strings, any set of characters between quotes. These can either be single quotes ' ' or double quotes " ".

Multiline strings must be surrounded by three quotations on each side of the text sequence.

```
a = """Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua."""
```

Strings are an example of a *sequence type*, meaning they store values in a specific order and allow individual elements to be accessed using an index.

3.7.1 Indexing Strings

Each character in a string has a position, called an *index*. Indexing in Python begins at 0, so the first character is at index 0. Think of it like asking the question for each letter: "How far from the start is this element?" The first element is zero positions away from the start, so its index is 0. The last element is always at index $n-1$, where n is the length of string. In memory, all of these characters are stored consecutively.

```
word = "Python"

print(word[0])
print(word[1])
print(word[5])
```

Output:

```
P
y
n
```

Attempting to access an index that does not exist will result in a runtime error.

3.7.2 Length of a String

The number of characters in a string can be found using the built-in `len()` function.

```
word = "Python"
print(len(word))
```

Output:

```
6
```

3.7.3 Iterating Over Strings

Strings can be iterated over one character at a time using a `for` loop.

```
for char in "Hello":
    print(char)
```

This loop runs once for each character in the string.

3.7.4 String Methods

Strings come with built-in functions, called *methods*, that perform common operations. These methods are called using dot notation.

```
text = "  Python Programming  "

print(text.upper())
print(text.lower())
print(text.strip())
```

Common string methods include:

- `upper()` converts all characters to uppercase
- `lower()` converts all characters to lowercase
- `strip()` removes leading and trailing whitespace

3.7.5 Lists

A list is another sequence type used to store multiple values in a single variable. Lists are created using square brackets, with elements separated by commas.

```
numbers = [1, 2, 3, 4]
```

Lists can store values of different datatypes, including strings, numbers, booleans, and even other lists.

```
data = [10, 3.14, True, "Python"]
```

3.7.6 Indexing Lists

Like strings, lists are indexed starting at 0.

```
numbers = [10, 20, 30]

print(numbers[0])
print(numbers[2])
```

Output:

```
10
30
```

For nested lists, you search an indexe using a bracket for every nested list until you reach the element you are aiming to reach. Later, we will talk about `.json` files, which are files essentially comprised of strung-nested lists.

```
nested_numbers = [0, [1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(numbers[0])
print(numbers[1][2])
print(numbers[2][1])
```

Output:

```
0
3
5
```

Slicing Strings

You can also use *slicing* in Python to get all elements between two indices. This syntax is Python-specific, although there are equivalences in other programming languages. This syntax works for *both* lists and strings.

For any sequence, we can slice it or operate on it with the following index notation:

```
sequence[start : stop : step]
```

where `start` is the index to begin at (inclusive), `end` is the index to end at (exclusive), and `step`, an optional argument for how many items to skip.

```
text = "abcdef"

text[1:4]    # 'bcd'
text[:3]     # 'abc'
text[3:]     # 'def'
text[::2]    # 'ace'
text[::-1]   # 'fedcba'  (reverse string)
```

The above example slices a string in a few different ways. Let's look at list slicing:

```
nums = [0, 1, 2, 3, 4, 5]

nums[2:5]    # [2, 3, 4]
nums[:4]     # [0, 1, 2, 3]
nums[4:]     # [4, 5]
nums[::2]    # [0, 2, 4]
nums[::-1]   # [5, 4, 3, 2, 1, 0]
```

Negative indices start from the end, and work the way up the sequence:

```
text = "python"

text[-3:]    # 'hon'
text[:-2]    # 'pyth'
```

Note that setting a variable to a sliced string creates a new string object, while leaving the original unchanged.

Exercise 11 Mystery Sequence

What will the code `seq[:]` output, assuming `seq = [a, f, j, k]` is a valid list?

Working Space

Answer on Page 95

Exercise 12 Computer... Indexing!

Working Space

You are given the following code:

```
1 word = "computer"
```

What is the output of each of the following expressions?

```
1 print(word[0])
2 print(word[2])
3 print(word[-1])
4 print(word[-3])
5 print(word[3:6])
```

Answer on Page 95

3.7.7 Modifying Lists

Unlike strings, lists are *mutable*, meaning their contents can be changed after creation.

```
1 numbers = [1, 2, 3]
2 numbers[1] = 10
3 print(numbers)
```

Output:

```
[1, 10, 3]
```

3.7.8 List Length and Iteration

The number of elements in a list can be found using the `len()` function.

```
numbers = [1, 2, 3]
print(len(numbers))
```

Lists can be iterated over using a `for` loop.

```
numbers = [1, 2, 3]

for n in numbers:
    print(n)
```

3.7.9 Common List Methods

Lists include built-in methods that allow elements to be added or removed.

```
numbers = [1, 2, 3]
numbers.append(4)
numbers.remove(2)
print(numbers)
```

Common list methods include:

- `append()` - adds an element to the end of the list
- `remove()` - removes the first occurrence of a value
- `pop()` – removes and returns an element by index

3.7.10 Strings vs Lists

Although strings and lists are both sequence types, there is an important difference between them.

- Strings are immutable and cannot be modified after creation
- Lists are mutable and can be changed

3.7.11 Summary

- Strings and lists are sequence types
- Indexing starts at 0
- The `len()` function returns the size of a sequence
- Strings are immutable
- Lists are mutable and support modification

Exercise 13 What's in your backpack?

Let's say the following code is run.

Working Space

```
1 items = ["pen", "book", "eraser"]
2
3 items.append("ruler")
4 items.pop()
5 items.remove("book")
6 items.append("marker")
```

Fill in the following chart with the contents of the `items` list after each operation.

Line Execution	`items` content
After Line 1	`items = ["pen", "book", "eraser"]`
After Line 3	
After Line 4	
After Line 5	
After Line 6	

Answer on Page 95

3.8 Is there more?

Of course, there is always something to learn with Python. We are going to do a deeper dive into functions, classes, and a few useful libraries in the next workbook. For now, this should be enough to get you started with Python. We will revisit many of these concepts in later chapters as we build more complex programs.

Chapter 4

Sets and Logic

The use of math usually falls into two categories:

- *Developing mathematical tools that let us make better predictions.* This is how engineers and scientists use math. It is usually referred to as *applied math*.
- *Creating interesting statements and proving them to be true or false.* This is known as *pure math*.

Many mathematical ideas start out as pure math, and eventually become useful. For example, the field of number theory is devoted to proving things about prime numbers. The mathematicians who created number theory were certain that it could never be used for any practical purpose. After a century or two, number theory was used as the basis of most cryptography systems.

Conversely, some ideas start out as a "rule of thumb" that engineers use, and are eventually rigorously defined and proven.

This course tends to emphasize applied math, but you should know something about the tools of pure math.

You can think of all the mathematical proofs as a tree. Each proof proves some statement true. To do this, the proof uses logic and statements that were proven true by other truths. So, the tree is built from the bottom up. However, the tree has to have a bottom. At the very bottom of the tree are some statements that we just accept as true without proof. These are known as *axioms*.

All of modern mathematics can be built from:

- A short list of axioms.
- A few rules of logic.

There have been several efforts to codify a small but complete axiomatic system. The most popular one is known as *ZFC*. "Z" is for the Ernst Zermelo, who did most of the work. "F" is for Abraham Fraenkel, who tidied up a couple of things. "C" is for The Axiom of Choice. As a community, mathematicians debate whether the Axiom of Choice should be an axiom; we get a couple of strange results if we include it in the system. If we do not, there are a few obviously useful ideas that we can't prove true.

ZFC has 10 axioms. We simply accept these 10 statements as true, and all the proofs of modern mathematics can be extrapolated from them. The 10 axioms are all stated in terms of sets. A few of them will be discussed below.

4.1 Sets

A *set* is a collection. For example, you might talk about the set of odd numbers greater than 5, or the set of all protons in the universe.

We have a notation for sets. For example, here is how we define S to be the set containing 1, 2, and 3:

$$S = \{1, 2, 3\}$$

We say that 1, 2, and 3 are *elements* of the set S. (Sometimes we will also use the word "member")

If you want to say "2 is an element of the set S" in mathematical notation, it is done like this:

$$2 \in S$$

If you want to say "5 is *not* an element of the set S", it looks like this:

$$5 \notin S$$

We have notation for a few sets that we use all the time:

Set	Symbol
The empty set	$\emptyset$
Natural numbers	$\mathbb{N}$
Integers	$\mathbb{Z}$
Rational numbers	$\mathbb{Q}$
Real numbers	$\mathbb{R}$
Complex numbers	$\mathbb{C}$

The empty set is the set that contains nothing. It is also sometimes called *the null set*.

Often, when we define a set, we start with one of these big sets and say "The set I'm talking about is the members of the big set, but only the one for which this statement is

true". For example, if you wanted to talk about all the integers greater than or equal to -5, you could do it like this:

$$A = \{x \in \mathbb{Z} \mid x \geq -5\}$$

When you read this aloud, you say "A is the set of integers x where x is greater than or equal to negative 5."

4.1.1 And and Or

Sometimes you need the members to satisfy two conditions; for this, we use "and":

$$A = \{x \in \mathbb{Z} \mid x > -5 \text{ and } x < 100\}$$

This is the set of integers that are greater than -5 *and* less than 100. In this book, we usually just write "and", but if you do a large amount of set and logic work, you will use the symbol $\wedge$:

$$A = \{x \in \mathbb{Z} \mid (x > -5) \wedge (x < 100)\}$$

Sometimes, you want a set that satisfies at least one of two conditions. For this, you use "or":

$$A = \{x \in \mathbb{Z} \mid x < -5 \text{ or } x > 100\}$$

These are the number that are less than -5 or greater than 100. Once again, there is a symbol for this:

$$A = \{x \in \mathbb{Z} \mid (x < -5) \vee (x > 100)\}$$

4.1.2 How simple are sets?

Sets are so simple that some questions just don't make any sense:

- "What is the first item in the set?" makes no sense to a mathematician. Sets have no

order.

- "How many times does the number 6 appear in the set?" makes no sense. 6 is a member, or it is not.

4.1.3 Subsets

If every member of set A is also in set B, we say that "A is a subset of B."

For example, if $A = \{1, 4, 5\}$ and $B = \{1, 2, 3, 4, 5, 6\}$, then A is a subset of B. There is a symbol for this:

$$A \subseteq B$$

Remember the table of commonly used sets? We can arrange them as subsets of each other:

$$\emptyset \subseteq \mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q} \subseteq \mathbb{R} \subseteq \mathbb{C}$$

Note that subsets have the transitive property: $\mathbb{N} \subseteq \mathbb{Z} \subseteq \mathbb{Q}$ thus $\mathbb{N} \subseteq \mathbb{Q}$

Note that if A and B have the same elements, $A \subseteq B$ *and* $B \subseteq A$. In this case, we say that the two sets are equal.

We also have a symbol for "is not a subset of": $A \not\subset B$

4.1.4 Union and Intersection of Sets

If you have two sets A and B, you might want to say "Let C be the set containing element that are in *either* A or B." We say that C is the *union* of A and B. There is notation for this too:

$$C = A \cup B$$

For example, if $A = \{1, 3, 4, 9\}$ and $B = \{3, 4, 5, 6, 7, 8\}$, then $A \cup B = \{1, 3, 4, 5, 6, 7, 8, 9\}$.

You also want to say "Let C be the set containing elements that are in *both* A and B." We say that C is the *intersection* of A and B. There is notation for this too:

$$C = A \cap B$$

For example, if $A = \{1, 3, 4, 9\}$ and $B = \{3, 4, 5, 6, 7, 8\}$, then $A \cap B = \{3, 4\}$.

4.1.5 Venn Diagrams

When discussing sets, it is often helpful to have a Venn diagram to look at. Venn diagrams represent sets as circles. For example, the sets A and B above could look like this:

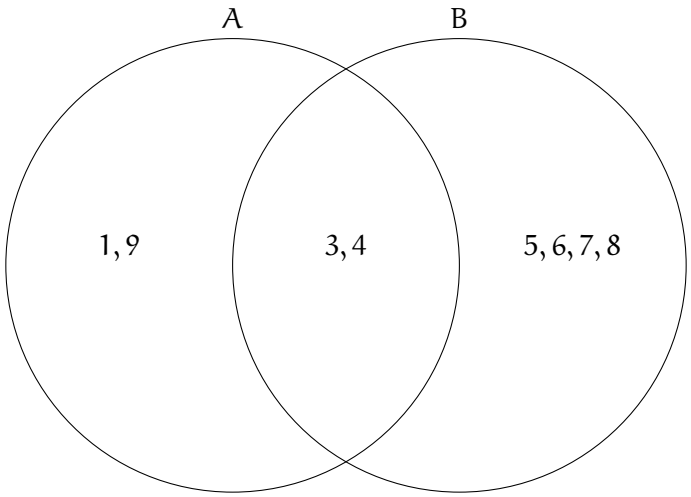


Figure 4.1: A Venn Diagram of the set of numbers $\{1, 9, 3, 4, 5, 6, 7, 8\}$

It makes it easy to see that A and B have a non-empty intersection, but they are not subsets of each other.

Often we won't even show the individual elements. For example, in the universe of all polygons, some rectangles are squares. Here's the Venn diagram:

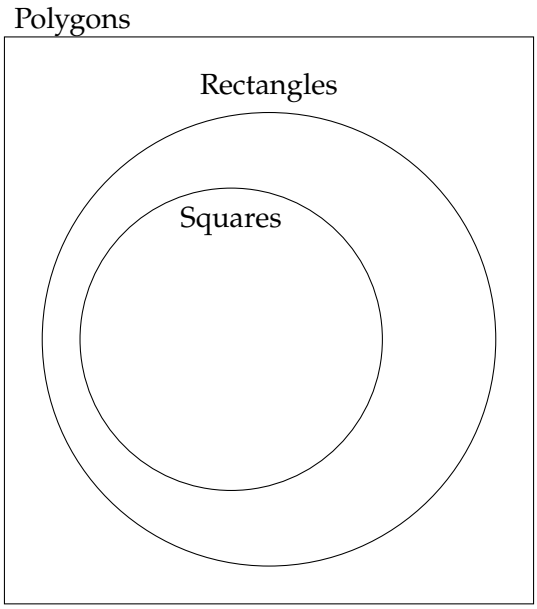


Figure 4.2: A Venn Diagram of polygons, rectangles, and squares.

As the combinations get more complex, we sometimes use shading to indicate what part we are talking about. For example, imagine we wanted all the rectangles with area greater than 5.0 that are not squares. The diagram might look like this:

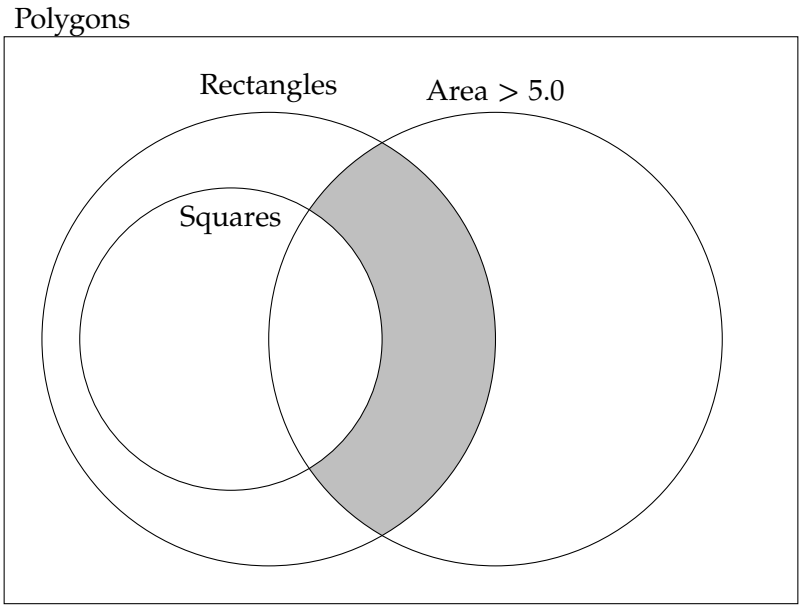


Figure 4.3: A Venn Diagram of the polygons, rectangle, and square number, and rectangles that have an area greater than 5.0 that are not squares.

4.2 Logic

We use a lot of logic in set theory. For example, the shaded region above represents all the polygons for which all the following are true:

- It is a rectangle.
- It is *not* a square.
- It has an area greater than 5.0.

4.3 Implies

In logic, we will often say "a implies b". That means "If the statement a is true, the statement b is also true." For example: "p is a square" implies "p is a rectangle".

There is notation for this: an arrow in the direction of the implication.

$$p \text{ is a square} \implies p \text{ is a rectangle}$$

Notice that implication has a direction: "p is a rectangle" does *not* imply "p is a square".

Implications can be chained together: If $A \implies B$ and $B \implies C$, then $A \implies C$.

4.4 If and Only If

If the implication goes both ways, we use "if and only if". This means the two conditions are equivalent. For example: "n is even if and only if there exists an integer m such that $2m = n$".

There is a notation for this too:

$$p \text{ is even} \iff \text{there exists an integer } m \text{ such that } 2m = n$$

There is even notation for "there exists". It is a backwards capital E, represented by $\exists$:

$$p \text{ is even} \iff \exists\, m \in \mathbb{Z} \text{ such that } 2m = n$$

4.5 Not

The not operation flips the truth of an expression:

- If a is true, not(a) is false.
- If a is false, not(a) is true.

We sometimes talk about "notting" or "negating" a value. We won't use it much, but there is a symbol for this: $\neg$.

We might create a *logic table* for negation that shows all the possible values and their negation:

A	¬A
F	T
T	F

This table says "If A is false, ¬A is true. If A is true, ¬A is false."

Most logic tables are for operations that take more than one input. For example, this logic table shows the values for and-ing and or-ing:

A	B	A and B	A or B
F	F	F	F
F	T	F	T
T	F	F	T
T	T	T	T

Notice that we have to enumerate all possible combinations of the inputs of A and B.

When a variable like A can only take two possible values, we say it is a *boolean* variable. (George Bool did important work in this area.)

Exercise 14 Logic Table

Working Space

Make a logic table that enumerates all possible combinations of boolean variables A and B and shows the value of the two following expressions:

- $\neg$ (A or B)
- ($\neg$A) and ($\neg$B)

Answer on Page 96

4.6 Cardinality

Informally, the *cardinality* of a set is the number of elements it contains. So, $\{1, 3, 5\}$ has a cardinality of 3. The null set has a cardinality of zero.

Things get a little trickier if a set is infinite. We say two infinite sets A and B have the same cardinality if there is some mapping that pairs every member of A with a member of B and mapping that pairs every member of B with a member of A.

For example, the set of all natural numbers is $\mathbb{N} = \{1, 2, 3, 4, \ldots\}$. The set of all even numbers is $\{2, 4, 6, 8, \ldots\}$. These have the same cardinality because we can pair each natural number n with an even number $2n$.

4.7 Complement of a Set

Most sets exist in a particular universe, for example you might talk about the even numbers as a set in the integers. You can then talk about the set's *complement*: the set of everything else. For example, the complement of the even numbers (inside the integers) is the odd numbers.

If you have a set A, its complement is usually denoted by A'.

See the Venn Diagram shown in Figure 4.4.

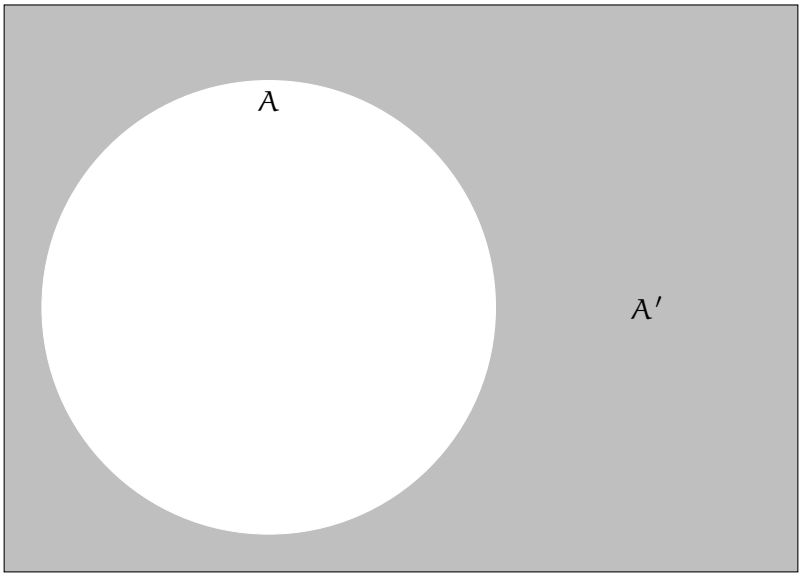


Figure 4.4: The complement of A is A'.

Outside of logic, the concept of a set

4.8 Subtracting Sets

If you have sets $A = \{1, 2, 3, 4\}$ and $B = \{1, 4\}$, it makes sense to subtract B from A by removing 1 and 4 from A.

If A and B are sets, we define $A - B$ to be $A \cap B'$. Take a second to look at this diagram and convince yourself that the white region represents $A - B$ and that it is the same as $A \cap B'$.

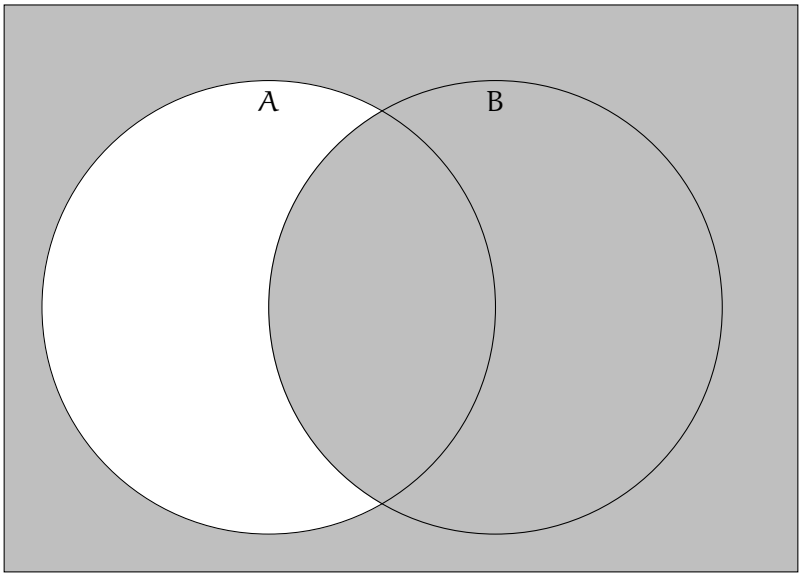


Figure 4.5: The subtraction of sets A and B.

4.9 Power Sets

It is not uncommon to have a set whose elements are also sets. For example, you might have the set that contains the following two sets: $\{1, 2, 3\}$ and $\{2, 3, 4\}$. You might write it like this: $\{\{1, 2, 3\}, \{2, 3, 4\}\}$. (Note that this set has a cardinality of 2 — it has two members that are sets.)

Given any set A, you can construct its *power set*, which is the set of all subsets of A. For example, if you have a set $\{1, 2, 3\}$, its power set is $\{\{1, 2, 3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1\}, \{2\}, \{3\}, \emptyset\}$.

If a set has n elements, its power set has 2^n elements. Note that the last element must be the empty set, because there are no subsets of an empty set. Recall that order does not matter in sets, so $\{1, 2\}$ and $\{2, 1\}$ are considered the same.

4.10 Booleans in Python

In Python, we can have variables hold boolean values: `True` and `False`. We also have operators: `not`, `and`, and `or`.

For example, you could find out what the expression "`a and not b`" is if both variables are false like this:

```
1 a = False
2 b = False
```

```
result = a or not b
print(f"a={a}, b={b}, a or not b = {result}")
```

This would print out:

```
a=False, b=False, a or not b = True
```

What if you wanted to try all possible values for a and b? You could use `itertools`.

```
import itertools

all_combos = itertools.product([False, True], repeat=2)
for (a, b) in all_combos:
    result = a or not b
    print(f"a={a}, b={b}: a or not b = {result}")
```

Type it in and run it. You should get the whole logic table:

```
a=False, b=False: a or not b = True
a=False, b=True: a or not b = False
a=True, b=False: a or not b = True
a=True, b=True: a or not b = True
```

If you had three inputs into the expression, your truth table would have eight entries. For example, if you wanted to know the truth table for $a and not (b and c)$, here is the code:

```
all_combos = itertools.product([False, True], repeat=3)
for (a, b, c) in all_combos:
    result = a and not(b and c)
    print(f"a={a}, b={b}, c={c}: a and not (b and c) = {result}")
```

Type it in and run it. You should get:

```
a=False, b=False, c=False: a and not (b and c) = False
a=False, b=False, c=True: a and not (b and c) = False
a=False, b=True, c=False: a and not (b and c) = False
a=False, b=True, c=True: a and not (b and c) = False
a=True, b=False, c=False: a and not (b and c) = True
a=True, b=False, c=True: a and not (b and c) = True
a=True, b=True, c=False: a and not (b and c) = True
a=True, b=True, c=True: a and not (b and c) = False
```

4.11 The Contrapositive

Here is a statement with an implication: "If it has rained in the last hour, the grass is wet."

This is *not* equivalent to "If the grass is wet, it has rained in the last hour." (After all, the sprinkler may be running.)

However, it is exactly equivalent to its *contrapositive*: "If the grass is not wet, it has not rained in the last hour."

The rule can be written using symbols:

$$(A \implies B) \iff (\neg B \implies \neg A)$$

4.12 The Distributive Property of Logic

Many ideas from integer arithmetic have analogues in boolean arithmetic. For example, there is a distributive property for booleans. These two expressions are equivalent:

- A and (B or C)
- (A and B) or (A and C)

So are these:

- A or (B and C)
- (A or B) and (A or C)

4.13 Exclusive Or

The expression "a or b" is true in any of the following conditions:

- a is True and b is False.
- a is False and b is True.
- Both a and b are True.

Sometimes engineers need a way to say "Either a or b is true, but not both." For this, we use *exclusive OR* (or XOR). You may see XOR written symbolically as $\oplus$ or just used as XOR. indexXOR

Here, then, is the logic table for XOR

A	B	XOR(a,b)
F	F	F
F	T	T
T	F	T
T	T	F

In Python, Logical XOR is done using `!=`:

```
1 just_one = (a != b)
```

(Take 10 seconds to confirm that this is the same as the logic table above.)

Chapter 5

Proofs

5.1 Introduction

Let's talk about *proofs*. In mathematics, a proof shows that a new statement aligns with all true statements which have come before it. A proof is a sequence of statements that starts with some assumptions and ends with a new, unproven conclusion. If the conclusion is true, then we say the proof is *valid*.

In mathematics, a proof shows that a new statement aligns with all true statements which have come before it.

The general structure will follow this pattern: We want to prove that $P \implies Q$ (this is said statement P *implies* statement Q).

We will assume some statement is true, and work to prove a different statement is true based on our assumptions and the rules of logic.

Here are some definitions to get us started:

- A *proposition* is a statement that is either true or false.
- A *theorem* is a proposition that has been proven to be true.
- A *lemma* is a *helper* proposition that is proven or known to be true and is used in the proof of a theorem.
- A *corollary* is a proposition that follows directly from a theorem.
- An *implication* is a theorem that relies on one proposition being equivalent to another.
- P is the *hypothesis* or *premise*.
- Q is the *conclusion*.
- The symbol $\forall$ represents the mathematical symbol for "for all", often used to iterate through all numbers of a subset.
- The symbol $\exists$ represents the mathematical symbol "there exists".
-

5.2 Trivial Proof

The easiest of proof is a **Trivial Proof**. If we know that Q is true always, we trivially know that P is true, and that P implies Q. This is most common when P and Q are unrelated statements with no affect on each other.

Theorem 1. *If* Q *is true, then* $P \implies Q$ *is true for any proposition* P.

Proof. Let P be the statement "The sky is blue" and Q be the statement "$2+2=4$". Since Q is always 1true, the implication $P \implies Q$ is true regardless of the truth value of P.

This follows directly from the definition of logical implication (or can be verified via a truth table), where an implication is true whenever the consequent is true. □

5.3 Direct Proof

A **Direct Proof** involves assuming the hypothesis (P) is true and then showing that the conclusion (Q) must also be true. This is done sequentially using logical deductions, equivalences, and previously established definitions.

To walk through an example, let's first define even and odd numbers.

Even and Odd Numbers

We say an integer n is *even* if there exists an integer k such that $n = 2k$.

We say an integer n is *odd* if there exists an integer k such that $n = 2k+1$.

Theorem 2. *The sum of two even integers is even.*

Proof. Let a and b be even integers. Then there exist integers k and m such that

$$a = 2k \quad \text{and} \quad b = 2m.$$

Consider their sum:

$$a + b = 2k + 2m = 2(k+m).$$

Since $k+m$ is an integer, it follows that $a+b$ is even. □

5.4 Proof By cases

A **proof by cases** is a technique where you split a problem into a finite number of distinct scenarios that cover all possibilities, then prove the statement in each scenario.

The structure is generally as follows:

1. Identify all possible cases (they must be exhaustive (cover all cases) and non-overlapping).
2. Prove the statement separately in each case. Usually, a direct proof is used in each case.
3. Conclude the statement holds in the general case.

Let's prove the following statement using a proof by cases:

Theorem 3. *For any integer n, n^2 is even or odd.*

Proof. Every integer is either even or odd. Therefore there are only two cases to consider:

Case 1. *n is even. Then, there exists an integer k such that $n = 2k$. Therefore,*

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2)$$

which is even.

Case 2. *n is odd. Then, there exists an integer k such that $n = 2k + 1$. Therefore,*

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$$

which is odd.

Since these cases cover all integers, the claim holds for all integers n. □

Exercise 15 Absolute Value Proof

Prove the following claim using a proof by cases:

Working Space

Theorem 4.

$$|x| = \begin{cases} x & x \geq 0 \\ -x & x < 0 \end{cases}$$

Answer on Page 96

5.5 Disproof By Counter-example

A proof by counterexample,more often called a **disproof by counterexample** is used to show a statement is false by finding one case or instance (a *counterexample*) where it fails.

Theorem 5. *All even numbers are divisible by* 4.

Proof. Consider the integer 2. It, by definition, is even, as $\exists\, k = 1$ s.t $2 \cdot 1 = 2$, which is even.

However, we must check if 2 is divisible by 4. An integer is divisible by 4 if it can be written in the form $4m$ for some integer m. Suppose, for purposes of disproving, that $2 = 4m$ for some integer m. Then

$$m = \frac{2}{4} = \frac{1}{2},$$

which is not an integer. Hence, 2 is not divisible by 4.

Thus, we have found an even integer that is not divisible by 4, contradicting the claim that *all* even integers have this property. Therefore, the statement is false. □

5.6 Proof By Contraposition

Proof by contraposition is a method used to prove an implication of the form

$$P \implies Q.$$

Instead of proving this statement directly, we prove its **contrapositive**:

$$\neg Q \implies \neg P.$$

The key idea is that an implication and its contrapositive are logically equivalent. We assume Q is false ($\neg Q$ is true), and directly show that P is false ($\neg P$ is true). If the negation of Q is true and the negation of P is true, it is equivalent to P and Q being valid:

$$\neg Q \implies \neg P \iff P \implies Q$$

This may make more sense with an example:

Theorem 6. *Let* n *be an integer. If* $3n + 2$ *is even, then* n *is even.*

Proof. Using proof by contraposition, we will prove the above theorem. Assume n is odd.

Then, $\exists\, k$ s.t $n = 2k + 1$. Then, through mathematical operations, Substitute into $3n + 2$:

$$\begin{aligned} 3n + 2 &= 3(2k + 1) + 2 \\ &= 6k + 3 + 2 \\ &= 6k + 5 \\ &= 2(3k + 2) + 1 \end{aligned}$$

which is odd.

Therefore, the contrapositive is true, and hence the original statement follows. □

5.7 Proof By Contradiction

Rationality states that you cannot believe two ideas which oppose each other at the same time. This is called a *contradiction*. For example, the statement "P is true and P is false" is a contradiction.

A proof by contradiction is a method of proving a statement by assuming the opposite of what you want to prove and then showing that this assumption leads to a contradiction or impossibility, represented by $\rightarrow\leftarrow$. This method is often used when direct proof is difficult or impossible. To prove the statement $P \implies Q$, we assume P is true and $\neg Q$ is true. We then show this leads to a contradiction ($\rightarrow\leftarrow$), which proves that if P is true, Q must also be true. One of these contradictions looks like:

- $1 = 0$ (or any similar false integer statement)
- $x \in A \wedge x \notin A$ (for set theory)
- $P \wedge \neg P$ (for logic)

We will use a lot of contradictory statements and proofs when we talk about thought experiments and paradoxes!

Theorem 7. *Let n be an integer. If $3n + 2$ is even, then n is even.*

Proof. Assuming $3n + 2$ is even. Further, for contradiction, assume n is odd. Then, $\exists\, k$ s.t $n = 2k + 1$. Then, through mathematical operations,

$$\begin{aligned} 3n + 2 &= 3(2k + 1) + 2 \\ &= 6k + 3 + 2 \\ &= 6k + 5 \\ &= 2(3k + 2) + 1 \end{aligned}$$

which is odd. This contradicts our assumption that $3n + 2$ is even. Therefore our assump-

tion that n is odd must be false, and n is even. □

5.8 Proof By Induction

A *proof by induction* is a methodology of proof that involves a (usually) proving a recursively defined statement. We use what is called the Principle of Mathematical Induction to assist with these proofs. We start with a simple case of the problem. Often, if we can prove the simplest case, we can prove all other cases by showing the each case depends on the previous one.

Recursion is useful in both mathematics and computer science because it provides a way to define a problem as correct by breaking it into smaller, similar subproblems. In mathematics, recursive definitions (such as sequences, factorials, or tree structures) are able to use proof techniques like induction, as their formulas rely on the previous term.

In computer science, recursion simplifies the implementation of algorithms that operate on data that can be broken down into smaller versions of itself, such as trees (which can be broken into smaller trees), graphs (which can be broken into subgraphs), and divide-and-conquer algorithms. We are going to cover recursive algorithms in depth in the Advanced Python Chapter. In general, there are 5 steps to this proof structure:

1. Define a specific proposition in the form of $P(n)$ *works correctly for all valid inputs of size* n. This is often given to you.
2. Define a *base case*. Just as in recursive definitions, proofs by induction us a base case as an "exit condition", or a start point to build off of. The base case proves that the proposition works for the smallest input size, which is usually $n = 0$ or $n = 1$. We call this $P(0)$.
3. Define an *inductive hypothesis*. Here, we assume the proposition remains correct for size k, some integer $k \geq 0$. We don't actually prove anything yet!
4. *Inductive Step*. Using the inductive hypothesis, prove $P(k + 1)$, which states that proposition is correct for size $k + 1$. Note here, that most times, the inductive step reduces the problem to a smaller, already proved case size. We say that proving $P(k + 1)$ is recursively defined by $P(k)$.
5. Conclude the proof by stating *By the Principle of Mathematical Induction, the proposition holds for all* $n \geq P(0)$, where $P(0)$ is the base case size. This step is optional but good practice.

Let's look at an example to clear things up!

Theorem 8. *For all integers* $n \geq 4$, $2^n < n!$. *Equivalently, for all integers* $n \geq 0$, $2^{n+4} < (n+4)!$.

Proof. We prove the statement by induction on n.

Basis Step. Let $n = 4$. This is our smallest input value within our subset. Then we have $2^4 = 16$ and $4! = 24$. $16 < 24$, so the proposition holds for the basis step.

Inductive Hypothesis. Suppose for some $k \in \mathbb{Z}_{\geq 4}$, $2^k < k!$.

Inductive Step. We want to show $P(k+1)$, such that $2^{k+1} < (k+1)$. Let $n = k+1$. Then we have

$$2^{k+1} = 2 \cdot 2^k$$

By the hypothesis,

$$2 \cdot 2^k < 2 \cdot k!$$

. Since $k \geq 4$, $k + 1 < 2$, making

$$2 \cdot k! < (k+1)k! = (k+1)!$$

. Therefore, by the PMI, the proposition holds for all $n \geq 4$. □

5.9 Summary

In this chapter we talked about mathematical proofs. A proof is a logical argument that starts at a proposition and leads to a conclusion $P \implies Q$. There are many proof methods, even some we did not talk about in this chapter, so make sure to watch addition videos for your own learning. The most important proofs techniques covered are:

- Direct Proofs, where we assume the starting statement P is true, and use logical to derive to Q
- Contraposition, where we prove $\neg Q \implies \neg P$, which is equivalent to $P \implies Q$
- Contradiction, where we prove by deriving impossibility from an assumption that the proposition could be false. This will come back in our paradoxes chapter!
- Induction, where a base case is used as a starting step to reach all other steps. These will also come back in my computer science chapters!

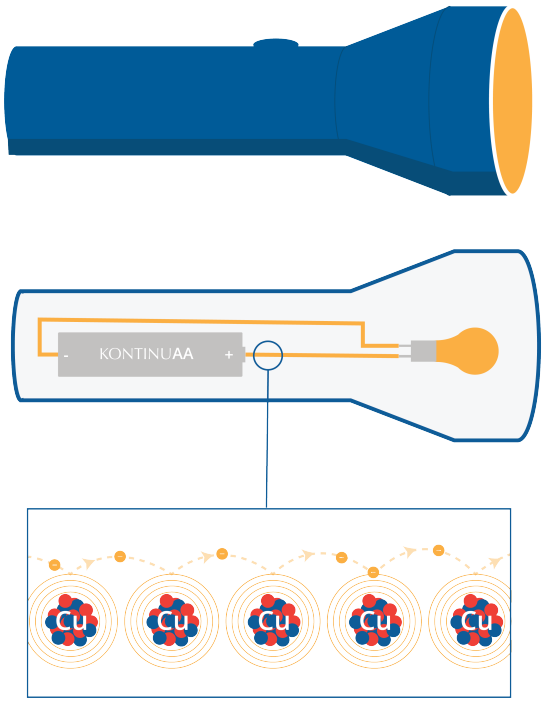
Overall, this chapter builds up many of the methodologies needed for proofs used to rigorously verify mathematical claims.

Chapter 6

Introduction to Electricity

What happens when you turn on a flashlight? The battery in the flashlight acts as an electron pump, and the electrons flow through the wires to the lightbulb (or LED). As the electrons pass through the lightbulb, they excite the molecules within, which gives off light and heat. (LEDs also give off light and heat, but they give off much less heat.) The electrons then return to the battery to be pumped around again.

When electricity is flowing through a copper wire, the protons and neutrons of the copper stay put, while the electrons jump between the atoms on their way from the battery to the lightbulb and back again.



In some materials, like copper and iron, electrons are loosely bound to their nuclei, forming a sea of electrons, which allows energy to flow. These are good *electrical conductors*. In the presence of an electric field, which we will talk about in a future chapter, the electrons are free to move about, allowing *charge* to flow free. In other materials, like glass and

plastic, electrons don't leave their nuclei as easily. This makes them are terrible electrical conductors – we call these materials *electrical insulators*. For example, the plastic around a wire is used for electrical insulation.

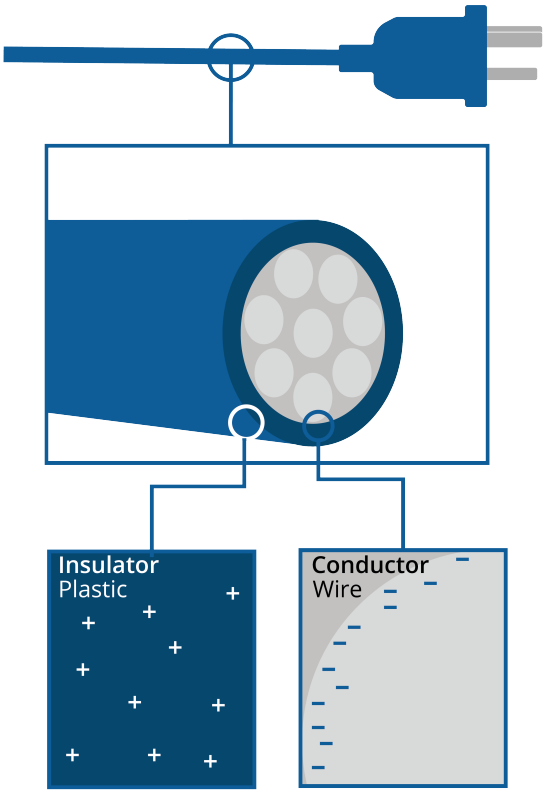


Figure 6.1: A cross section of an outlet plug.

6.1 Units

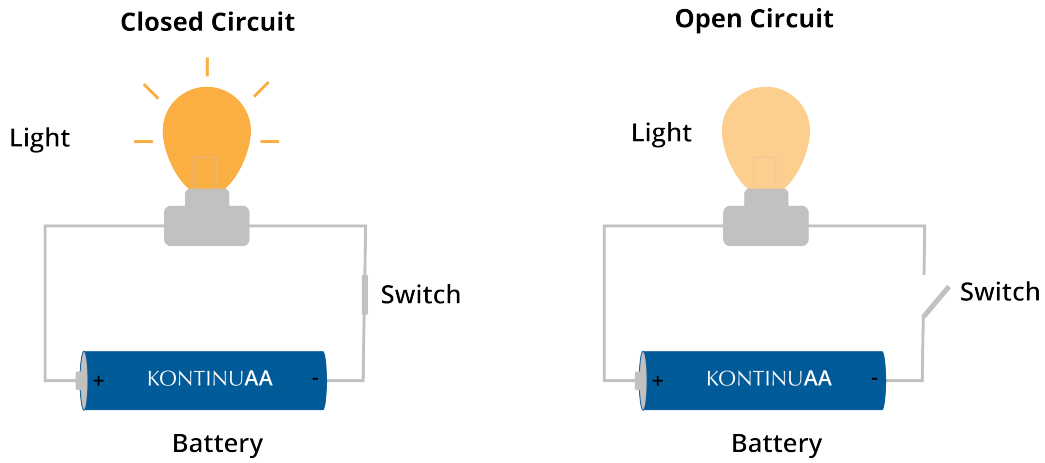
Electrons are very small, so to study them, scientists came up with a unit that represents *a lot* of electrons. 1 *coulomb* is about 6,241,509,074,460,762,608 electrons. When 5 coulombs enter one end of the wire every second (and simultaneously 5 coulombs exit the other end), we say "This wire is carrying 5 amperes of current."

(Truthfully, we usually shorten ampere to just "amp". This is sometimes a little awkward, because we also often shorten the word "amplifier" to "amp", but you should generally be able to tell which is which from the context.)

If you look at the circuit breakers or fuses for your home's electrical system, you'll see that each one is rated in amps. For example, maybe the circuit that supplies power to your kitchen has a 10 amp circuit breaker. If, for some reason, more than 10 amps tries to pass

through that wire, the circuit breaker will turn off the whole circuit.

When your flashlight is on, it pushes about 1 amp of current through the lightbulb (When it is off, there is no current in the lightbulb).



The lightbulb creates *resistance* that the current pushes through. Think of it like plumbing: The current is the amount of water passing through a pipe. The resistance is something that tries to stop the current – like a ball of hair, similar how different surfaces apply friction when pushing a box. The battery is what allows the current to push through the resistance; we call that pressure *voltage*. Especially in physics, voltage is often referred to as *electromagnetic potential*.

Superconductor is an extreme example of a conductor. Here, there is *absolutely no resistance* to the passing of charge. Materials that are conductors can become superconductors by decreasing their temperature to extremely low temperatures, even to absolute zero.

6.2 Capacitors

Two conductors separated by a distance that carry equal but opposite charges ($+Q$ and $-Q$) are a system that we refer to as **capacitors**. Most real-life capacitors (like the cylindrical electrolytic ones) are actually long strips of foil rolled up. We treat them as parallel plates, or metal plates with d between them.

Capacitors are similar to batteries, in the way that they store energy, however, capacitors store energy in the form of an electric field. As a result, capacitors store much less energy but can release it at a significantly higher rate. The measure of how much a capacitor

holds is called **capacitance**, represented with the symbol C and measured in Farads (F), where 1 F $= \frac{1 \text{ Coulomb}}{1 \text{ Volt}}$.[1]

Capacitance (C) is defined as the ratio of the magnitude of the charge (Q) on either conductor to the potential difference (V) between them:

$$C = \frac{Q}{V}$$

We will explore the underlying nature of charge in greater detail in the Charge chapter. For a parallel-plate capacitor with a vacuum between the plates ($\kappa = 1$), the capacitance is determined by its geometry:

$$C = \epsilon_0 \frac{A}{d}$$

Think of an medical defibrillator, a machine that delivers a specific amount of volts within an instantaneous moment of time. The goal of a defibrillator is not just to provide *charge*, but to provide Energy (U_C) to *reset* the heart's electrical rhythm. The energy stored is given by:

$$U_C = \frac{1}{2} C V^2$$

which comes from the area under a **Voltage** (V) vs **Charge** Q, and the fact that potential charge is $Q = CV$

Defibrillators often charge to several thousand volts. Because V is squared in the energy equation, increasing the voltage is the most efficient way to increase the *shock* power. To keep the device portable while maintaining high U_C or Electric Potential Energy, manufacturers use advanced dielectrics (κ) to maximize C without making the internal plates massive. When the paddles touch the chest and the button is pressed, the capacitor discharges through the patient's body. The patient's chest acts as a resistor in an RC circuit. The current I follows

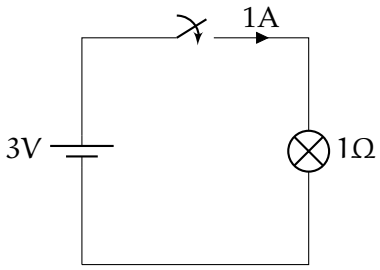
$$I(t) = \frac{V_0}{R} e^{-t/RC}$$

this is a great example of current following an example of exponential decay!

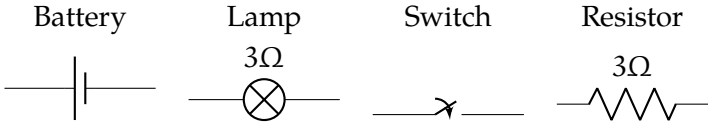
6.3 Circuit Diagrams

Here is a circuit diagram of your flashlight:

[1] 1 Farad is actually an enormous amount, enough to store power for the stereo system of an SUV! Smartphone circuits are 0.1010 μF, measured in microfarads, which are 10^{-6} F



The lines are wires. The symbols that we will use are:



The battery pushes the electrons from the positive end (the larger line) to the negative end (the smaller line), so the circuit must go around in a circle for the current to flow. This is why the current stops flowing when the switch breaks the circuit.

You can think of a switch as having zero resistance when it is closed and infinite resistance when it is open.

For our purposes, a lamp is just a resistor that gives off light.

6.4 Ohm's Law and Resistance

Current is a measure of how much *charge* per *time* is flowing through an area:

$$I = \frac{\Delta q}{\Delta t} \tag{6.1}$$

This is a measure of $1\frac{\text{coulomb}}{\text{second}}$, which we give the SI unit amperes or amps, represented with A.

Resistance is measured in *ohms*, and we use a Greek capital omega for that: Ω

Ohm's Law

Whenever a voltage V is pushing a current I through a resistance of R in given by someΩ, the following is true:

$$V = IR$$

where V is in volts, I is in amps, and R is in ohms.

Due to this relationship, we can find current given voltage and resistance using $I = \frac{V}{R}$. *Ohmic* materials maintain a constant resistance no matter how many volts are being put into it.

The resistance of a conductor depends on it's length, cross-sectional area, and resistivity. Due to Ohm's equation, we define resistance, measured in ohm's, as $1 \frac{\text{volt}}{\text{ampere}}$

6.5 Power and Watts

Joule's Law

When a current I is passing through a resistance R, the power consumed is

$$W = I^2R$$

where W is in watts, I is in amps, and R is in ohms.

Of course $V = IR$, so we can extend this to:

$$W = I^2R = IV = \frac{V^2}{R}$$

Your flashlight's batteries provide about 3 volts. How much battery power is the flashlight using when it is on? The power (in watts) produced by the battery is the product of the voltage (in volts) and the current (in amps). This means your flashlight is giving off $3volts \times 1amp = 3watts$ of power. Some of that power is given off as light, some as heat.

A watt is 1 joule of energy per second. We say that a watt is a measure of *power*.

When we talk about how much energy is stored in a battery, we use a unit like a kilowatt-hour. A kilowatt-hour is equivalent to 3.6 million joules.

Portable Chargers

A portable charger labeled 165 W means that it's *maximum power output* is 165 watts. This is the maximum amount of voltage it can deliver to your device at any given moment.

Often, there is a noted voltage and current label on the back of the device.Power is the product of Voltage (V) and Current (I), expressed by the formula:

$$P = V \times I$$

In the case of a 165 W charger, it might output something like 20 V at 8.25 A $(20 \times 8.25 = 165)$.

However, the 165 Watt charger will not always deliver 165 Watts to your phone. That is enough to power a gaming PC! Instead, protocols within modern chargers limit how much a device actually gets charged. Most smartphones only need 20 W. Chargers that are labeled fairly high in wattage often have multiple ports (this one has 4), and splits the charging load among all of them.

Now, if you see a measurement in mAh (milliampere-hours) or Wh (Watt-hours), that is the maximum *capacity* of the portable charger. This 165 W has a 25,000 mAh rating, meaning it stores approximately 92.5 Wh, or watt-hours. This comes from $\text{Wh} = \frac{\text{mAh} \times \text{V}}{1000}$, where most power banks deliver a nominal 3.7 V.[2]

6.6 Another great use of RMS

In many electrical problems, the voltage fluctuates a great deal. For example, the fluctuations in voltage makes the sound that comes out of an audio speaker.

You can use the root-mean-squared of the voltage to figure out the average power your speaker is consuming.

Let's say that the RMS of the voltage you are sending to the speaker is V_{rms} and the resistance of the speaker is R ohms. This means the power consumed by the speaker is:

$$P = \frac{V_{rms}^2}{R}$$

Similarly, if you know the RMS of the current you are pushing through the speaker is I_{rms}, then the power consumed by the speaker is:

$$P = I_{rms} R$$

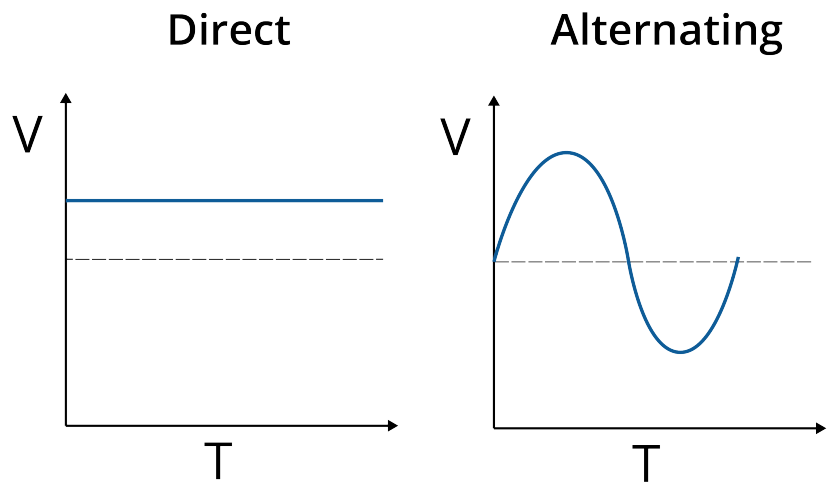
[2] Airlines often do not let you take anything over 100 Wh.

6.7 Electricity Dangers

Large amounts of electricity moving through your body can hurt or even kill you. You must be careful around electricity.

That said, your body is not a very good conductor, so low-voltage systems (like a flashlight) don't have enough voltage to move significant amounts of current through your body.

However, the electricity in a power outlet has much more voltage. The voltage in these outlets is fluctuating between positive and negative, so we call it *Alternating Current* or AC.



In most countries, the RMS of the voltage between 110 and 240 V. (The peak voltage is always $\sqrt{2}$ times the RMS value. In the US, for example, people say "Our outlets supply 120 V." They mean that the RMS of the voltage difference between the wire and the earth is 120V. The peak voltage is almost 170V.)

How much current can a human handle? Not much. You can barely feel 1 mA moving through your body, but at 16 mA, your muscles will clench and you won't be able to relax them — many people die from electrocution because they grab a wire which pushes enough current through their body to prevent them from letting go of the wire. At 20 mA, a human's respiratory muscles become paralyzed.

The fuse breaker in a house will often allow 20 A to flow through the circuit before it shuts off the power. Always be very sure to shut off the power before touching any of the

wiring in your house.

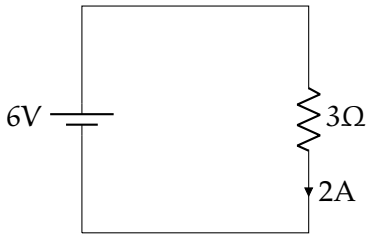
While water is actually a mediocre conductor, it can still deliver enough current to kill you. If you see a wire in a puddle, you should not touch the puddle. Interestingly, because of the salt, sea water is more than 100 times better at conducting electricity than the water you drink.

If you hold a wire in each hand, how many Ohms of resistance will your body have? Once it gets past your skin, you will look like a bag of salt water to the electricity. After the skin, your body will have a resistance of about 300Ω. However, the skin is a pretty good insulator. If you have dry, calloused hands, your skin may add 100,000Ω to the resistance.

CHAPTER 7

DC Circuit Analysis

In the most basic circuit, you have only a battery and a resistor:

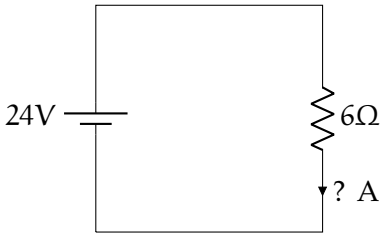


For this situation, you only need Ohm's Law: $V = IR$. In this case, $6V = 3\Omega \times 2A$.

Exercise 16 Ohm's Law

Working Space

How many amps are going around the circuit?

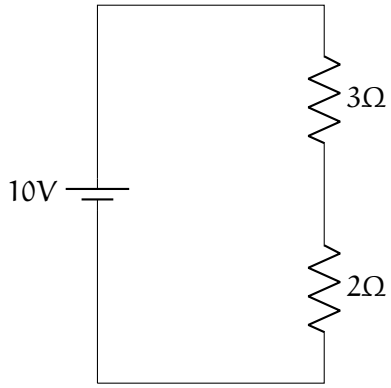


Answer on Page 96

7.1 Resistors in Series

When you have two resistors wired together in a long line, we say they are "in series." If you have two resistors R_1 and R_2 wired in series, the total resistance is $R_1 + R_2$.

In this diagram, for example, the total resistance is 5Ω.



The current flowing through the circuit, then, is $10/5 = 2A$.

By Ohm's law, the voltage drop across the upper resistor is $IR = 2A \times 3\Omega = 6V$.

The voltage drop across the lower resistor is $IR = 2A \times 2\Omega = 4V$.

7.1.1 Kirchhoff's Voltage Law

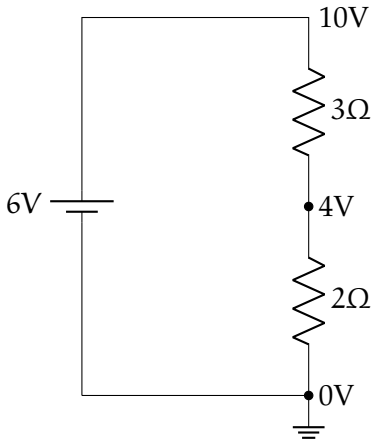
Notice that the battery pumps the voltage up to 10V, then the two resistors drop it by exactly 10V. This is known as "Kirchhoff's Voltage Law":

Kirchhoff's Voltage Law

As you make a loop around a circuit, the sum of the voltage increase must equal the sum of the voltage decrease. Mathematically, this is

$$\sum_{i=1}^{n} V_i = 0 \tag{7.1}$$

The negative end of the battery is connected to "ground" (it has zero voltage). We can then draw a diagram with the voltages (That symbol in the lower right represents a connection to ground).

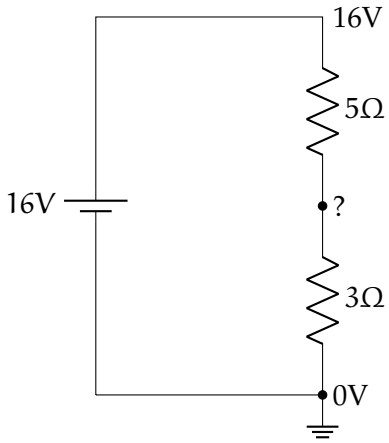


Exercise 17 Resistors In Series

Working Space

What is the current going around the circuit?

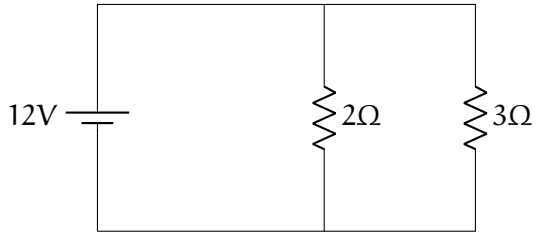
What is the voltage drop across each resistor?



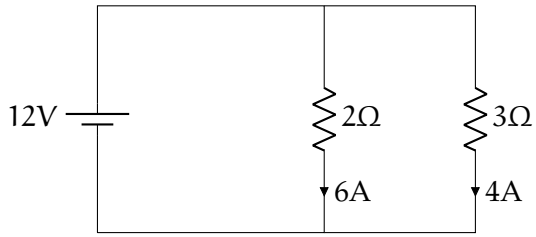
Answer on Page 96

7.2 Resistors in Parallel

Observe the following circuit. Note that the current can go two different paths.



There is 12 volts pushing current through both resistors. So 6A will go through the 2Ω resistor and 4A will go through the 3Ω resistor. Note that even though there is a path of least resistance (2Ω), the current is still divided evenly among both branches.



Thus, a total of 10 A will be going through the battery.

Imagine you are a battery. You can't see that you have two resistors. What does it feel like to you? $\frac{V}{I} = R$, and $V = 12$ and $I = 10$. This means the effective resistance of the two resistors in parallel is $\frac{12}{10}$ or $\frac{6}{5}\Omega$.

Resistance in Parallel

If you have several resistances $R_1, R_2, \ldots, R_n$ wired in parallel, their effective resistance R_t is given by

$$\frac{1}{R_t} = \frac{1}{R_1} + \frac{1}{R_2} + \ldots + \frac{1}{R_n}$$

In our example:

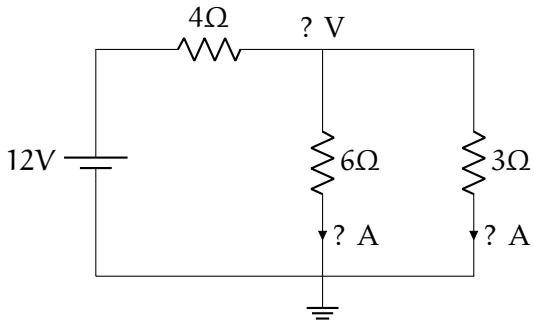
$$\frac{1}{R_t} = \frac{1}{2} + \frac{1}{3} = \frac{5}{6}$$

Thus, $R_t = \frac{6}{5}\Omega$.

Exercise 18 Resistors In Parallel

Working Space

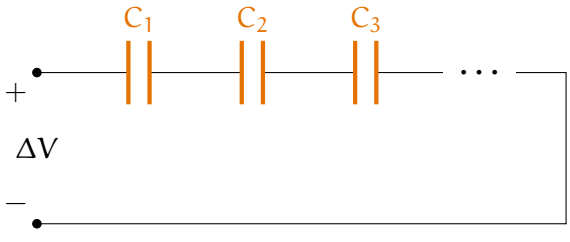
What is the current going through the battery? What is the drop over the 4Ω resistor? What is the current in each branch?



Answer on Page 97

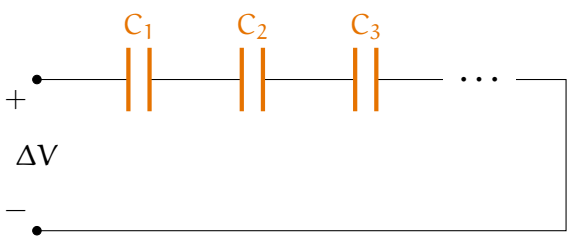
7.3 Capacitors in series and parallel

Recall that a capacitor is similar to a battery in that it stores voltage, but it is stored in the form of an electric field. Capacitors are represented with two differently colored lines. Here is an circuit diagram of capactors in **parallel** Capacitors are said to be in parallel if they share the same potential difference. You may see this represented mathematically:



$$C_p = \sum_i C_i \tag{7.2}$$

Capacitors are **series** if they all share the same charge magnitude. This would be



mathematically represented as

$$\frac{1}{C_S} = \sum_i \frac{1}{C_i} \tag{7.3}$$

A common mistake in calculating series capacitance is neglecting to take the reciprocal to find the equivalent capacitants after finding the reciprocal of each individual capacitor. If you have n capacitors in series all with value C, the total capacitance is $C_S = \frac{C}{n}$.

7.4 Kirchhoff's Current Law

Kirchhoff's Current Law states that all currents coming into a junction (or node) must equal the currents leaving that junction.

Why? Because first of all, energy is always conserved, meaning it cannot be lost or destroyed within the equations in the first place.

Secondly, each circuit is conservative, meaning it cannot lose voltage, and thus, cannot lose current. As you go around a loop, the starting point must have the same chargeable amount as it did when you began, so any increases and decreases along the loop have to cancel out for a total change of zero.

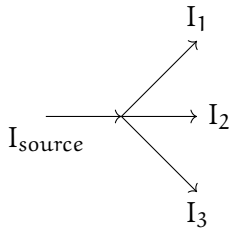
Kirchhoff's Current Law

The sum of current into a junction equals the sum of current out of the junction.

Mathematically, this can be expressed as

$$\sum_{i=1}^{n} I_i = 0 \tag{7.4}$$

where n is the total number of branches with currents flowing towards or away from the node.



Kirchhoff's Current Law states that all currents going out of a junction must equal the current going into a junction. This is aligned with the Conservation of energy: **the total energy of an isolated system remains constant**. In this case,

$$I_{source} = I_1 + I_2 + I_3 \tag{7.5}$$

Exercise 19 Kirchhoff's Current Law

At a junction, 5 conductors meet at a single common junction. Given Branch Currents:

- $i_1 = 5$ A (entering the node)
- $i_2 = 3$ A (leaving the node)
- $i_3 = 2$ A (entering the node)
- $i_4 = 9$ A (leaving the node)

Find the direction of the unknown current i_x.

Answer on Page 97

7.5 Summary

This chapter introduces the fundamentals of Direct Current (DC) circuits, focusing on the relationship between power sources and resistive elements.

- **Basic circuits**: The analysis begins with a minimal circuit configuration consisting of a voltage source (battery) and a load (resistor).
- **Voltage** (V): The electrical potential difference provided by the battery that drives the flow of charge.
- **Resistance** (R): The property of a material or component that opposes or *resists* the flow of electric current.
- **Direct Current** (DC): A type of electrical flow where the charge moves in a single, constant direction.
- **Kirchhoff's Laws**: Kirchhoff's Laws tell us that charge cannot be lost or pile up at a junction, and any supplied charged must be dropped to zero before the charge cycles around.

In a few workbooks, we will be doing a deep dive into charge, and how charge creates electric fields, electric flux, dipoles, and spherical electric potential.

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 6)

$$\mu = \frac{1}{6}(87 + 91 + 98 + 65 + 87 + 100) = 88$$

Answer to Exercise 2 (on page 8)

The mean of your grades is 88.

The variance, then, is

$$\sigma^2 = \frac{1}{6}\left((87-88)^2 + (91-88)^2 + (98-88)^2 + (61-88)^2 + (87-88)^2 + (100-88)^2\right) = \frac{784}{6} = 65\frac{1}{3}$$

The standard deviation is the square root of that: $\sigma = 8.083$ points.

Answer to Exercise 3 (on page 9)

In order the grades are 65, 87, 87, 91, 98, 100. The middle two are 87 and 91. The mean of those is 89. (Speed trick: The mean of two numbers is the number that is halfway between.)

Answer to Exercise 4 (on page 22)

The formula for the RMS is "=SQRT(SUMSQ(A2:A1001)/COUNT(A2:A1001))".

Answer to Exercise 5 (on page 34)

Variable	Value	Data Type
a	10	`int`
b	3.5	`float`
c	”10”	`str`
d	True	`bool`
e	[1, 2, 3]	`list`

Answer to Exercise 6 (on page 35)

```
False
```

Answer to Exercise 7 (on page 37)

Line Executed	n (value, type)	m (value, type)
After line 2	(5, int)	(”3”, str)
After line 3	(6, int)	(”3”, str)
After line 4	(6, int)	(”31”, str)

Answer to Exercise 8 (on page 39)

The code will raise a `TypeError` because `x` is a string and cannot be added to the integer `5`. The error message will be similar to:

```
TypeError: can only concatenate str (not "int") to str
```

To fix this, we need to cast `x` to an integer using `int()`:

```
x = int(input("Enter a number: "))
print(x + 5)
```

Answer to Exercise 9 (on page 44)

Input	Output
5	Small
10	Ten
15	Large

Answer to Exercise 10 (on page 44)

```
for i in range(0, 21, 2):
    print(i)
```

Answer to Exercise 11 (on page 49)

The start index is omitted and the end index is omitted, which means they are 0 and `len(seq)`$= 4$ respectively. Since the bounds are 0 and $4 - 1 = 3$, every index is included, in the given order. Therefore, `seq[:]` returns a copy of the given string, `[a, f, j, k]`

Answer to Exercise 12 (on page 49)

```
c
m
r
t
put
```

Answer to Exercise 13 (on page 51)

Line Execution	`items` content
After Line 1	`items = ["pen", "book", "eraser"]`
After Line 3	`items = ["pen", "book", "eraser", "ruler"]`
After Line 4	`items = ["pen", "book", "eraser"]`
After Line 5	`items = ["pen", "eraser"]`
After Line 6	`items = ["pen", "eraser", "marker"]`

Answer to Exercise 14 (on page 61)

A	B	not (A or B)	(not A) and (not B)
F	F	T	T
F	T	F	F
T	F	F	F
T	T	F	F

Notice that the two expressions are equivalent!

DeMorgan's Rule says "not (A or B)" is equivalent to "(not A) and (not B)".

It also says "not (A and B)" is equivalent to "(not A) or (not B)".

Answer to Exercise 15 (on page 69)

Proof.

Case 3. *Suppose* $x \geq 0$. *By the definition of absolute value,* $|x|$ *represents the distance of* x *from* 0 *on the real line. Since* x *is nonnegative, this distance is simply* x. *Hence,* $|x| = x$. 0

Case 4. *In this case,* x *is negative, so its distance from* 0 *is given by its additive inverse. Thus,* $|x| = -x$.

□

Notice how explicit we are being with our proof, using words and mathematical syntax.

Answer to Exercise 16 (on page 85)

$V = IR$ so $I = \frac{V}{R} = \frac{24V}{6\Omega} = 4A$.

Answer to Exercise ?? (on page 87)

There is a total resistance of 8Ω, so your 16V will push 2A of current around the circuit.

2A going through a 5Ω resistor represents a 10V drop.

2A going through a 3Ω resistor represents a 6V drop.

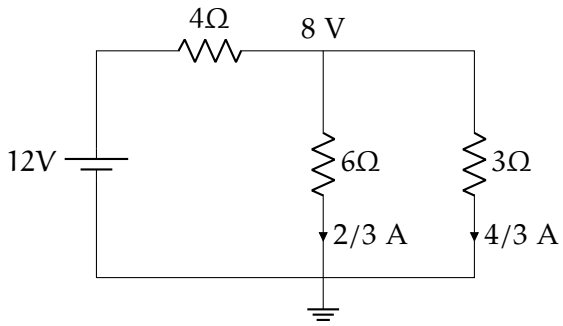
Answer to Exercise 18 (on page 89)

The effective resistance of the 6Ω and the 3Ω is 2Ω because

$$\frac{1}{R_T} = \frac{1}{6} + \frac{1}{3} == \frac{1}{2}$$

This means the battery experiences a resistance of $4\Omega + 2\Omega = 6\Omega$. A 12V will push 2A through a resistance of 6Ω.

The voltage drop across the 4Ω resistor is $2A \times 4\Omega = 8V$. Thus there will be a 4V drop across the two resistors in parallel. So 2/3 A will flow through the 6Ω resistor. 4/3 A will flow through the 3Ω resistor.



Answer to Exercise 19 (on page 91)

Let's set up the equation. Here are the entering conductors:

$$i_1, i_3$$

And leaving: i_2, i_4, i_x we are assuming i_x is leaving for the calculation.

$$i_1 + i_3 = i_2 + i_4 + i_x$$

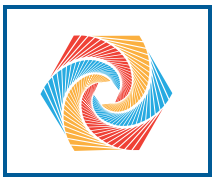
Inputting the values gives us:

$$5 + 2 = 3 + 9 + i_x$$

$$7 = 12 + i_x$$

$$i_x = -5\,A$$

The magnitude of the current is 5 A. Because the result is negative, the actual direction is *entering* the node, based on our leaving assumption.



Index

∅, 54
ℂ, 54
ℕ, 54
ℚ, 54
ℝ, 54
ℤ, 54

addition, 35
amp or ampere, 76
and, 55
applied math, 53
augmented assignment operator, 36
axioms, 53

bell curve, 10
boolean, 60
boolean variables, 63
booleans, 33
break, 42

capacitors, 77
    in parallel, 89
    in series, 89
cardinality, 61
casting, 38
complement, 61
conclusion, 67
conditionals, 40
conservation of energy, 90
contradiction, 71
contrapositive, 65
corollary, 67
coulombs, 76
counterexample, 70

defibrillator, 78
dictionary, 33
disproof by counterexample, 70
division, 35

elements, 54
even, 68
exclusive OR, 65

floats, 32
floor division, 35
for loops, 41

histograms, 9
hypothesis, 67

if and only if, 59
implication, 67
implies, 59
index, 45, 86
inductive hypothesis, 72
Inductive Step, 72
input, 37
installing python, 29
integers, 32
intersection, 56

Joule's law, 80

Kirchhoff's current law, 90
Kirchhoff's voltage law, 86

lemma, 67
logic, 59
logic table, 60
loops, 40

match-case, 40, 43
mean, 5
median, 8
modulus, 35
multiplication, 35

normal distribution, 10
not, 60

odd, 68
Ohm's law, 79
ohms, 79
operations, 35
or, 55

power set, 63
premise, 67
print, 31
proof by cases, 68
proof by induction, 72
proofs, 67
proposition, 67
pure math, 53

quadratic mean, 11

recursion, 72
resistance, 77
    in parallel, 88
RMS, 11
root-mean-squared, 11

samples, 5
sequence types, 33
set, 54
sets of sets, 63
Spreadsheet
    Entering formula, 17
spreadsheet, 15
    graphs, 25
string methods, 46
strings, 32, 44
    indexing, 45
    iterating, 46
    length, 45
subset, 56
subtracting sets, 62
subtraction, 35
summation symbol, 6
symbolic vs. numeric solutions, 15

the null set, 54
theorem, 67

union, 56

valid, 67
variance, 6
Venn diagrams, 57
voltage, 77
volts, 79

watts, 80
while loops, 42

ZFC, 53